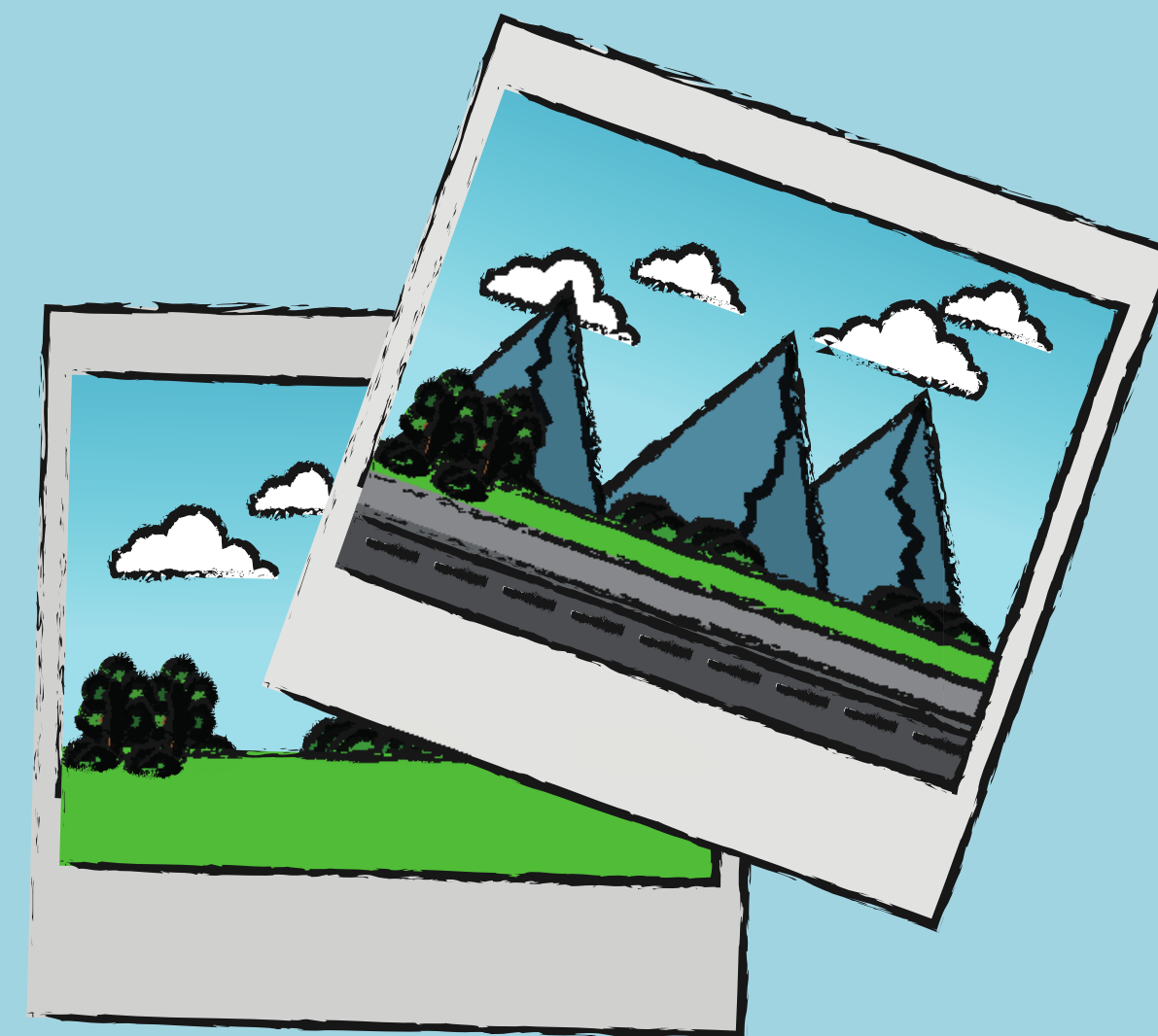


Môn học: Học máy | GVHD: Phan Đình Vần

BÁO CÁO CUỐI KỲ

IMAGE CAPTIONING



NHÓM 4



NỘI DUNG BÁO CÁO



Cơ sở lý thuyết



Bộ dữ liệu và demo



Đạo đức trong AI

CƠ SỞ LÍ THUYẾT



TRÌNH TẠO CHÚ THÍCH ẢNH

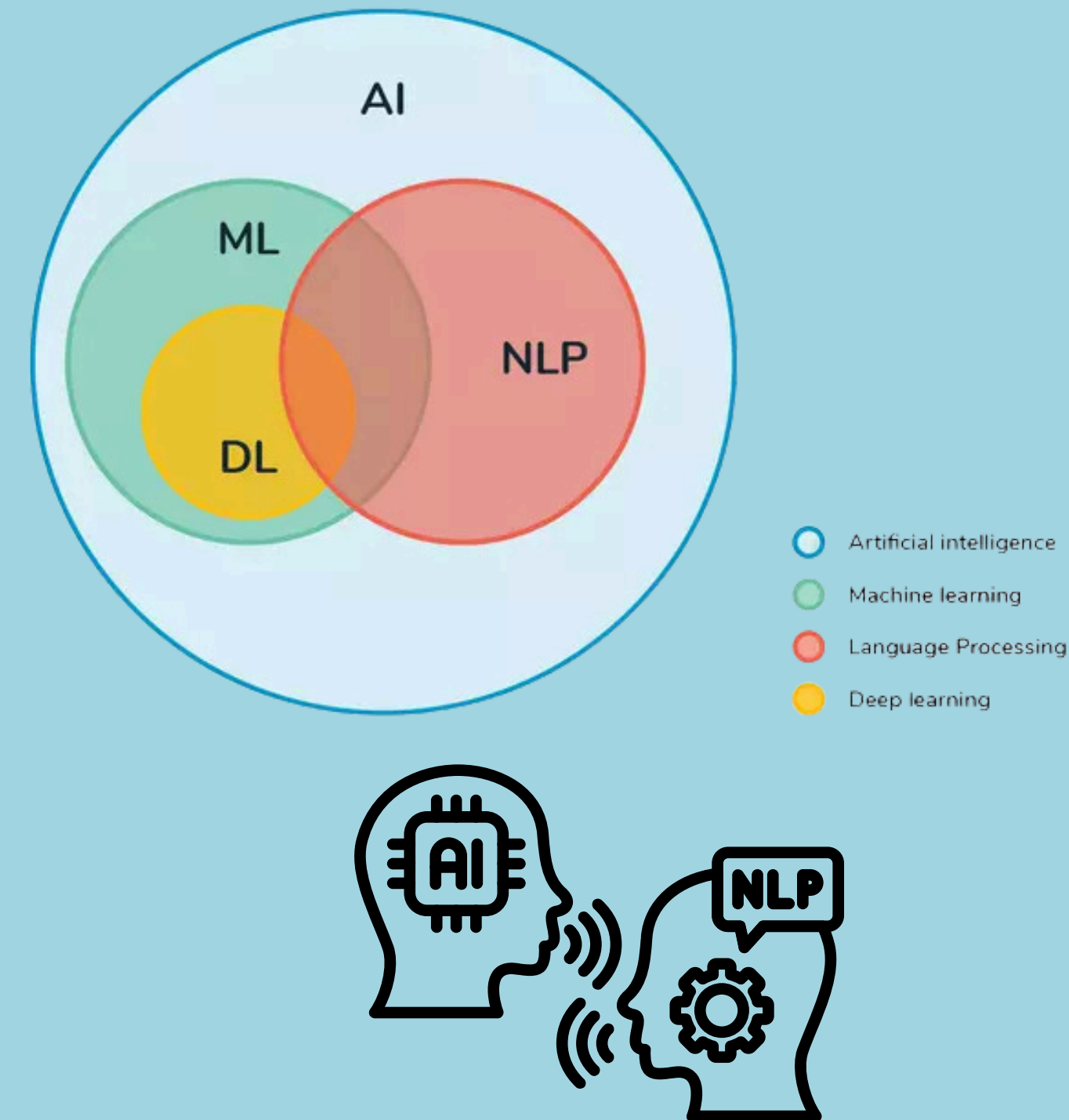
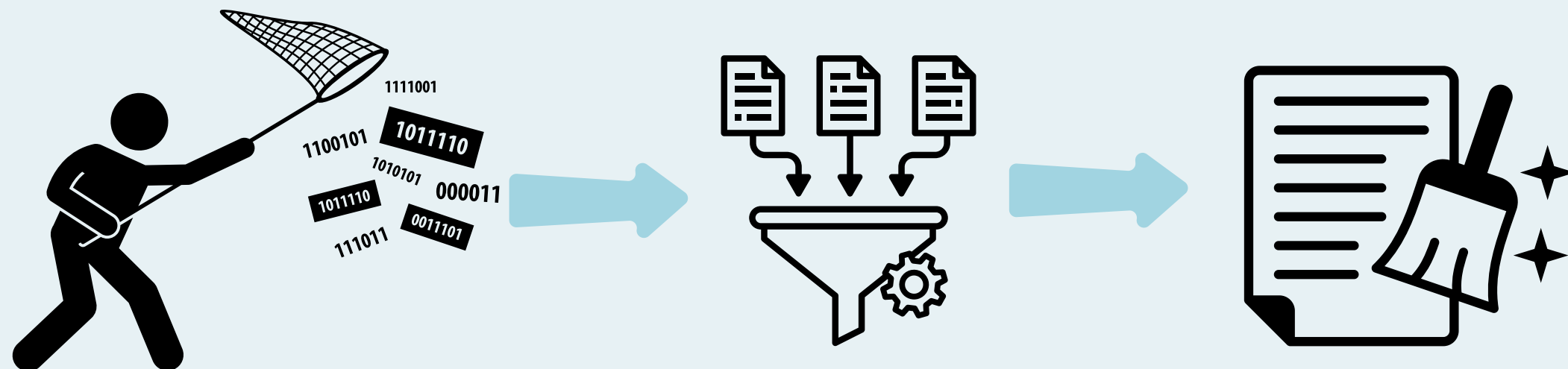
Là một nhiệm vụ liên quan đến thị giác máy tính và các khái niệm xử lý ngôn ngữ tự nhiên để nhận biết ngữ cảnh của hình ảnh và mô tả chúng bằng ngôn ngữ tự nhiên.



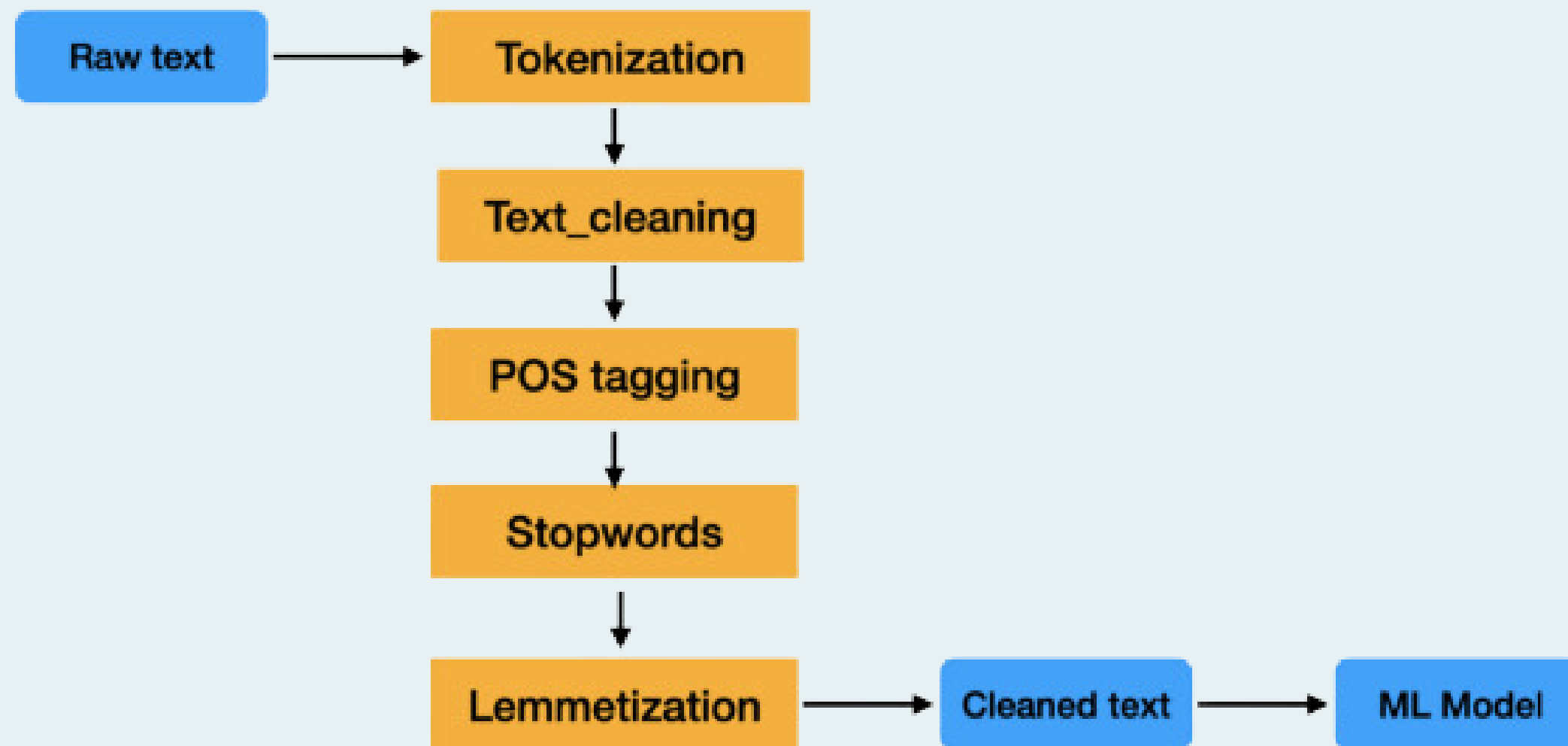
XỬ LÝ NGÔN NGỮ TỰ NHIÊN (NATURAL LANGUAGE PROCESSING)

Bảo Phương

- Là một nhánh nghiên cứu của trí tuệ nhân tạo, trong đó tập trung vào việc cho phép máy tính **hiểu và xử lý được ngôn ngữ của con người**.
- Tiền xử lý văn bản: là 1 bước trong xử lý ngôn ngữ tự nhiên giúp **loại bỏ các yếu tố gây nhiễu** để tạo ra một tập dữ liệu sạch, nhất quán, dễ tiếp cận và có cấu trúc rõ ràng hơn cho các mô hình học máy.



QUY TRÌNH TIỀN XỬ LÝ TRONG NLP



- Chuyển văn bản thành chữ thường
- Loại bỏ URL
- Xóa dấu câu
- Xóa chữ số
- Tách câu
- Chuyển biểu tượng cảm xúc thành văn bản (Emojis and Emoticons)
- Xóa các từ không có nghĩa (Stop Words)
- Chuẩn hóa văn bản (Standardizing Text)
- Sửa lỗi chính tả (Correcting Spelling)
- Tách từ (Tokenizing)
- Chuẩn hóa từ (Stemming and Lemmatization)

TIỀN XỬ LÝ TRONG DỰ ÁN TẠO CHỦ THÍCH ẢNH

1

Tách từ
(tokenize)

2

Chuyển sang chữ
thường (lowercase)

3

Loại bỏ dấu câu
khỏi từng từ
(như ., ?, !, v.v.)

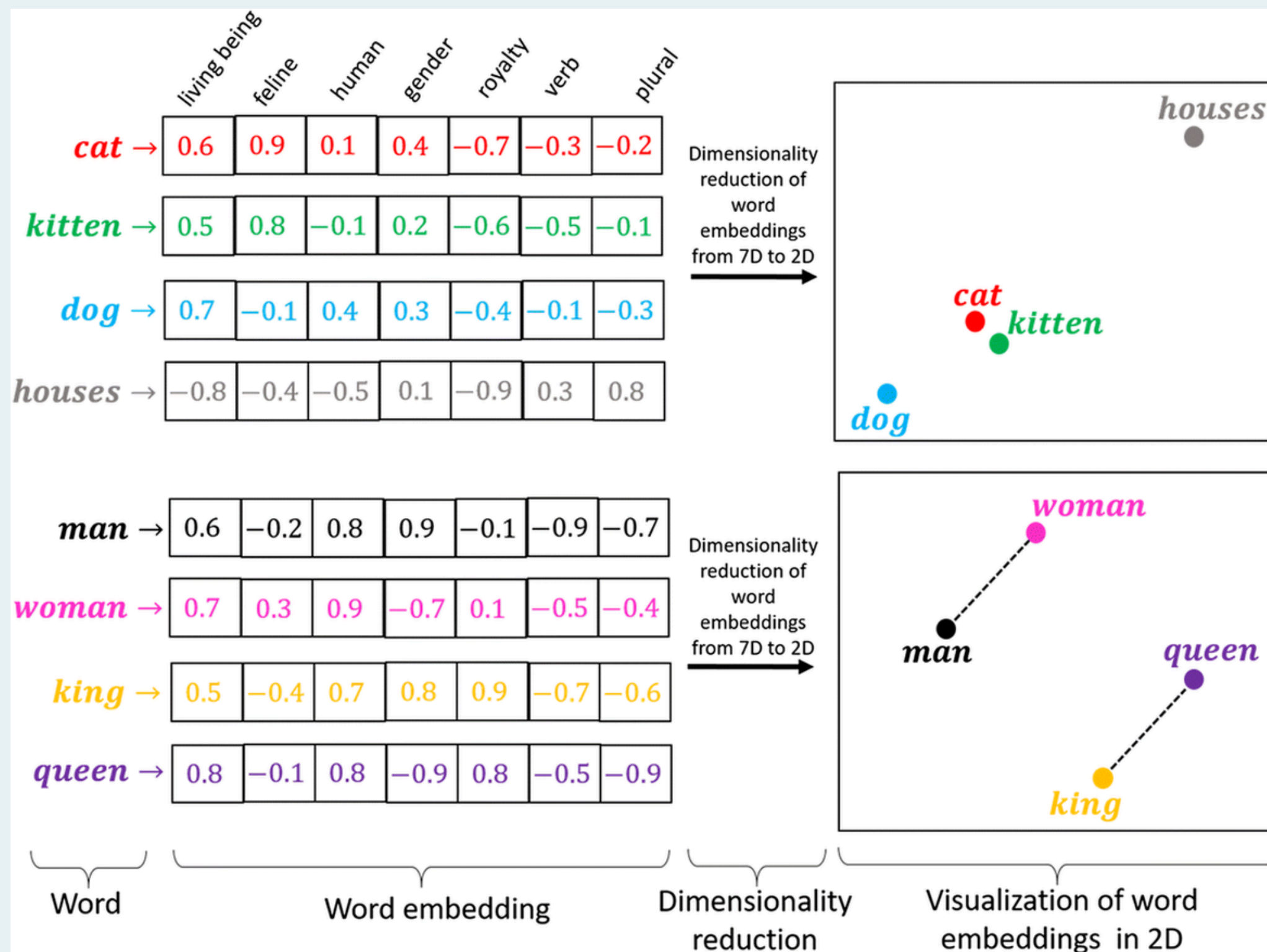
4

Loại bỏ từ có độ dài
ngắn: Các từ có độ
dài ≤ 1 (như s hoặc a)

5

Loại bỏ từ chứa số

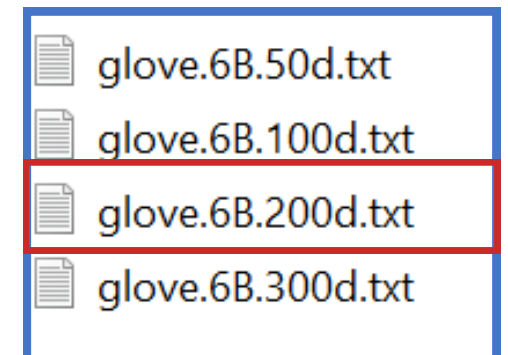
WORD EMBEDDING



- Word Embedding là kĩ thuật để **biểu diễn từ dưới dạng các vecto số học**, từ đó miêu tả được mối liên hệ, sự tương đồng về mặt ngữ nghĩa, văn cảnh (context) của dữ liệu.
- Vecto này bao gồm nhiều chiều và các từ trong không gian đó mà có **cùng văn cảnh hoặc ngữ nghĩa** sẽ có vị trí gần nhau.

WORD EMBEDDING TRONG DỰ ÁN TẠO CHÚ THÍCH ẢNH

- Sử dụng **GloVe (Global Vectors for Word Representation)**: một mô hình embedding từ được huấn luyện trước, cung cấp các biểu diễn vector số cho từ vựng. Nó được huấn luyện trên các tập dữ liệu lớn như Wikipedia hoặc Common Crawl để học mối quan hệ ngữ nghĩa giữa các từ.
- File glove.6B.200d.txt chứa các vector embedding 4 kích thước 200 cho từng từ.
- Mỗi dòng là một từ và vector 200 chiều biểu diễn ngữ nghĩa của từ đó.
- Quy trình:



Nạp file
embedding
GloVe

Tạo ma trận
embedding
(embedding_matrix)

Khởi tạo lớp embedding
(embedding_layer)

CHUYỂN GIAO HỌC TẬP (TRANSFER LEARNING)

- Là một kỹ thuật trong học sâu, trong đó một mô hình đã được huấn luyện trước trên một tập dữ liệu lớn (ví dụ: ImageNet) được tái sử dụng và tùy chỉnh cho một bài toán mới
- Các bước chính trong Transfer Learning:

Sử dụng mô hình đã huấn luyện trước

- Mô hình được huấn luyện trước (pre-trained model) thường được tải từ các thư viện như TensorFlow, PyTorch, hoặc Keras.

Đóng băng (freeze) các tầng đã huấn luyện trước

- Không cập nhật trọng số trong quá trình huấn luyện của các tầng đầu để giữ nguyên các đặc trưng đã học.

Tinh chỉnh (Fine-tune)

- Mở khóa một số tầng ở cuối của mô hình và huấn luyện lại với dữ liệu mới.

Thay đổi tầng đầu ra (Output layer)

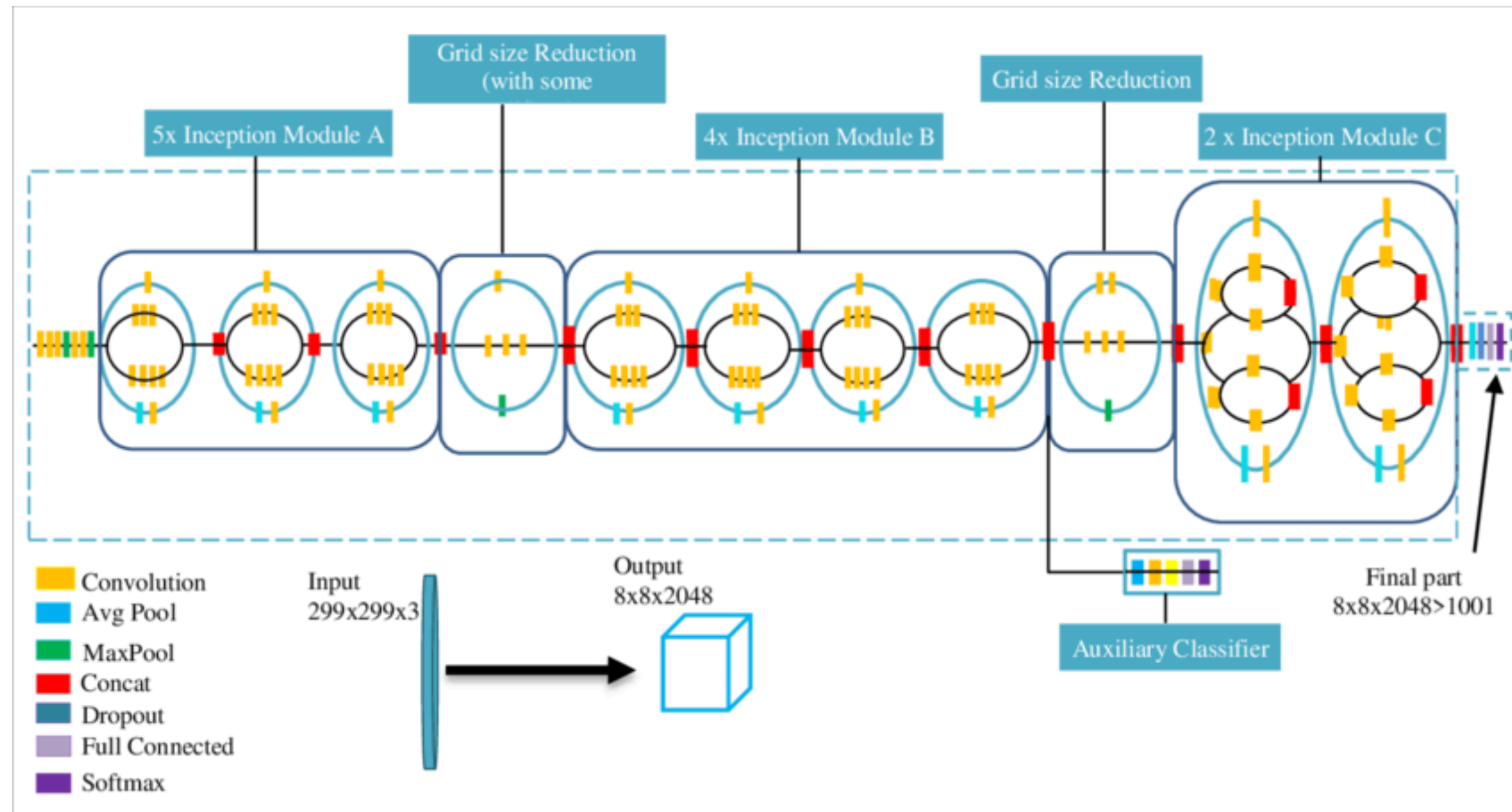
- Thay thế tầng đầu ra bằng một hoặc nhiều lớp phù hợp với số lượng nhãn của bài toán.

- Trong dự án nhóm, nhóm sẽ Loại bỏ lớp cuối cùng của **Inception v3** để sử dụng mô hình như một công cụ trích xuất đặc trưng của ảnh.

INCEPTION V3

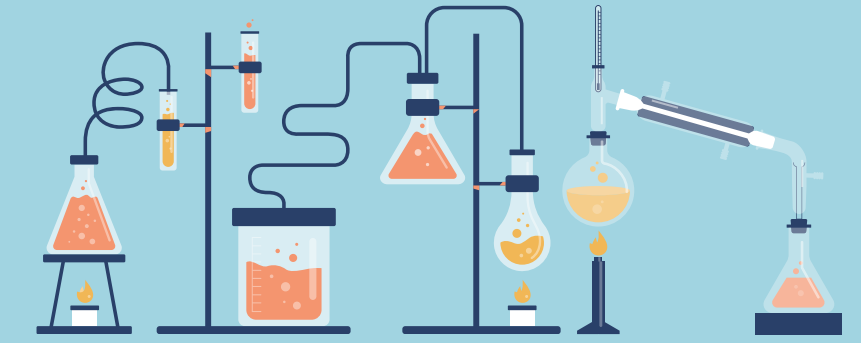
- Inception v3 là một **mô hình mạng nơ-ron tích chập** (Convolutional Neural Network - CNN) được thiết kế cho **các tác vụ nhận dạng hình ảnh và phân loại hình ảnh**. Đây là phiên bản thứ ba trong chuỗi các mô hình Inception, được phát triển bởi nhóm nghiên cứu của Google
- Inception v3 có độ sâu khoảng 42 lớp => Giúp mô hình học được các đặc trưng phức tạp hơn từ dữ liệu hình ảnh.

CẤU TRÚC CỦA INCEPTION V3

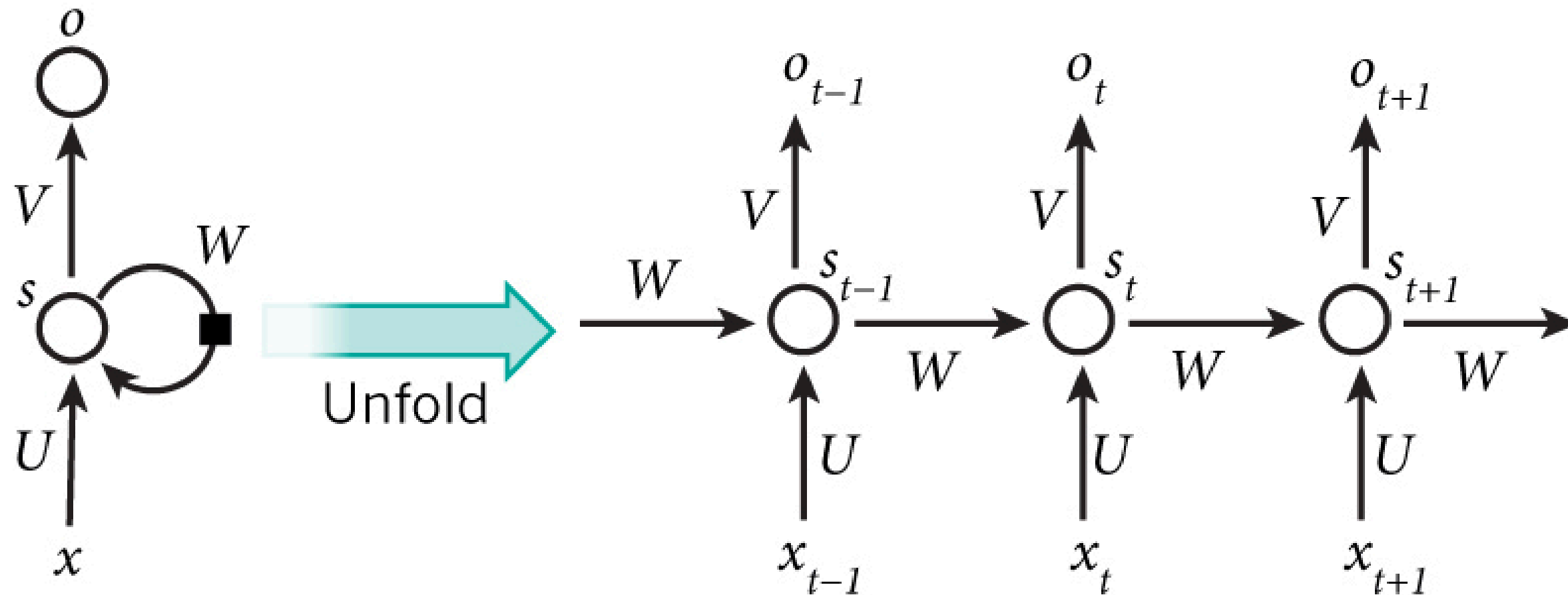


- Sử dụng các khối Inception (Inception blocks), trong đó nhiều kích thước bộ lọc (filters) được **áp dụng song song**, giúp mạng có khả năng trích xuất đặc trưng đa dạng (từ cục bộ đến tổng thể).
- Các khối này **kết hợp cả bộ lọc 1×1 , 3×3 , 5×5** , và phép pooling để tối ưu hóa việc trích xuất đặc trưng mà không làm tăng quá nhiều số tham số.

LSTM (RNN)



Ý tưởng chính của RNN (Recurrent Neural Network) là sử dụng chuỗi các thông tin. RNN có khả năng nhớ các thông tin được tính toán trước đó



Được tạo thành từ các nơ-ron: các nút xử lý dữ liệu kết hợp cùng nhau để thực hiện các tác vụ phức tạp. Các nơ-ron được tổ chức dưới dạng

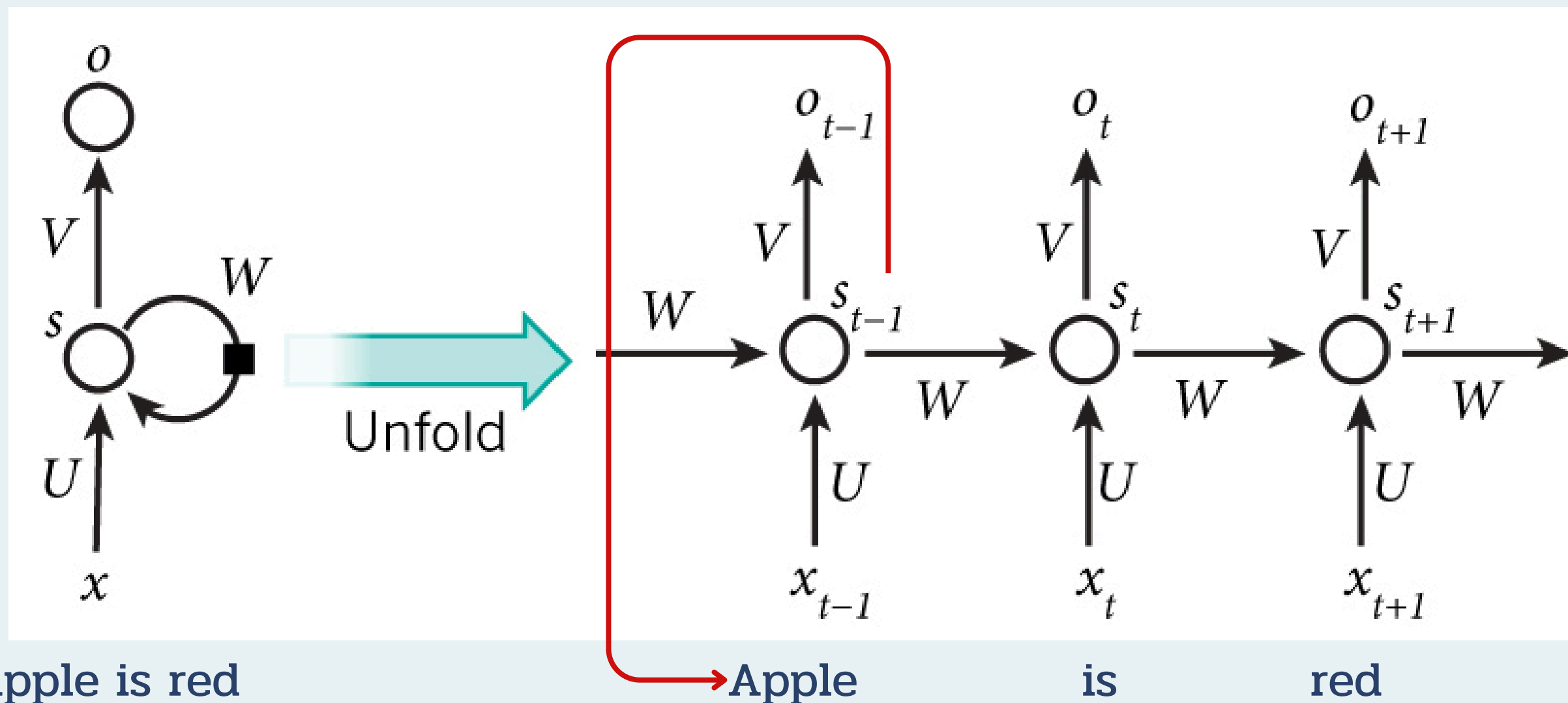
- Lớp đầu vào: nhận thông tin để xử lý
- Lớp ẩn : xử lý dữ liệu, phân tích và dự đoán
- Lớp đầu ra: cung cấp kết quả

LSTM (RNN)



RNN hoạt động bằng cách lần lượt truyền dữ liệu tuần tự nhận được đến các lớp ẩn.

Lớp ẩn có thể ghi nhớ và sử dụng các đầu vào trước đó cho các dự đoán trong tương lai trong một thành phần bộ nhớ ngắn hạn (sử dụng đầu vào hiện tại và bộ nhớ đã lưu trữ).



Ứng dụng RNN

- Hệ thống dịch máy (Machine Translation).
- Tự động sinh văn bản

LSTM (RNN)



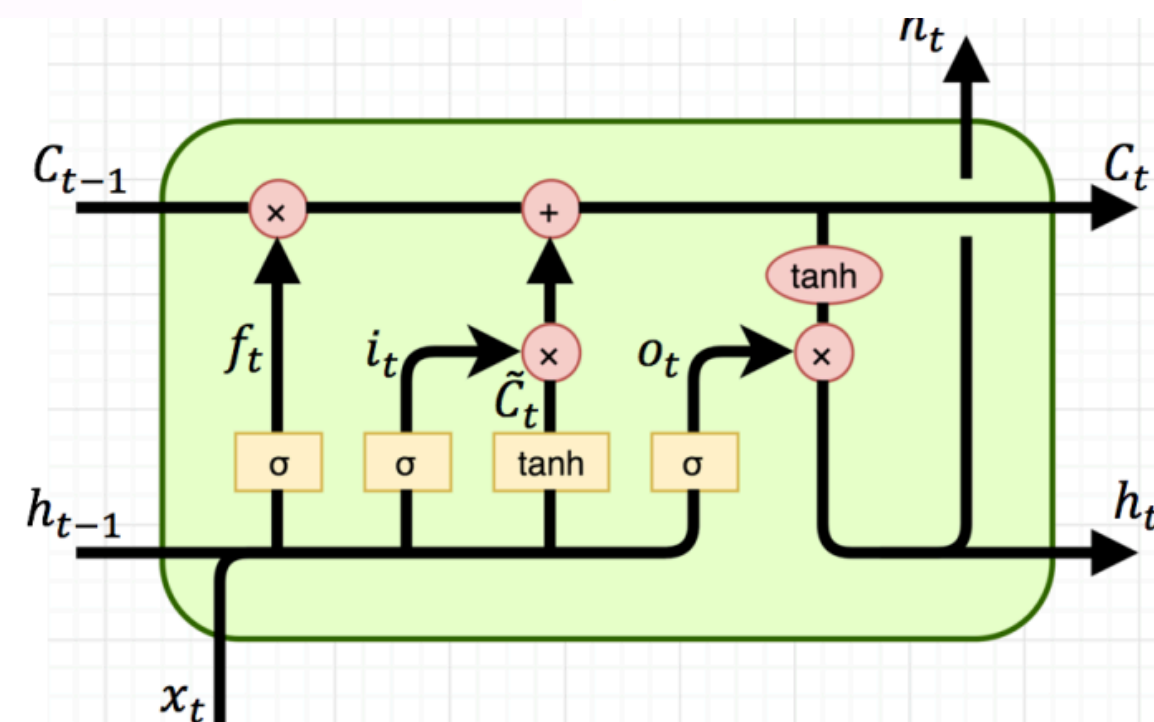
LSTM là một dạng đặc biệt của mạng nơ-ron hồi tiếp (RNN) được thiết kế để **giải quyết vấn đề vanishing gradient** trong việc huấn luyện các mô hình học sâu trên dữ liệu chuỗi dài.

LSTM có khả năng nhớ các thông tin lâu dài nhờ các "cổng" đặc biệt, cho phép nó duy trì thông tin trong một khoảng thời gian dài.

RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

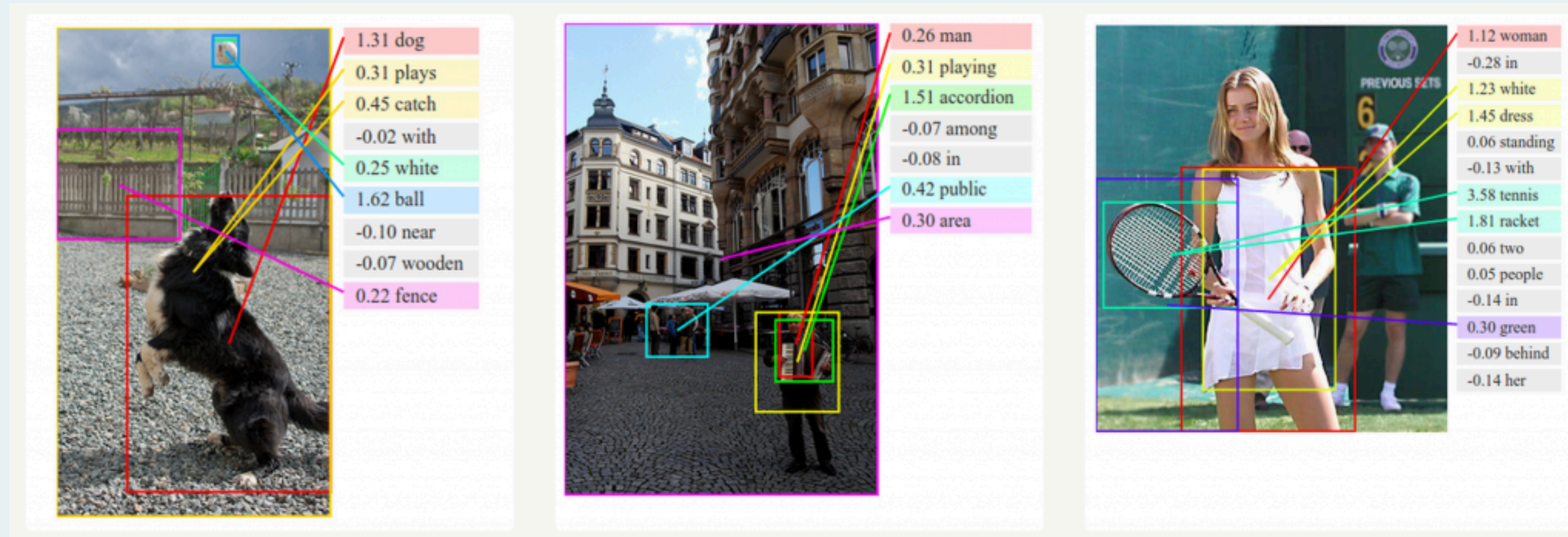


$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

- Rất phù hợp với bài toán sinh văn bản (text generation).

ỨNG DỤNG LSTM (RNN)

Các mô tả cho mỗi hình ảnh được chuẩn bị dưới dạng các câu, và LSTM sẽ học cách dự đoán các từ tiếp theo trong câu, dựa vào các đặc trưng của hình ảnh đã được trích xuất.

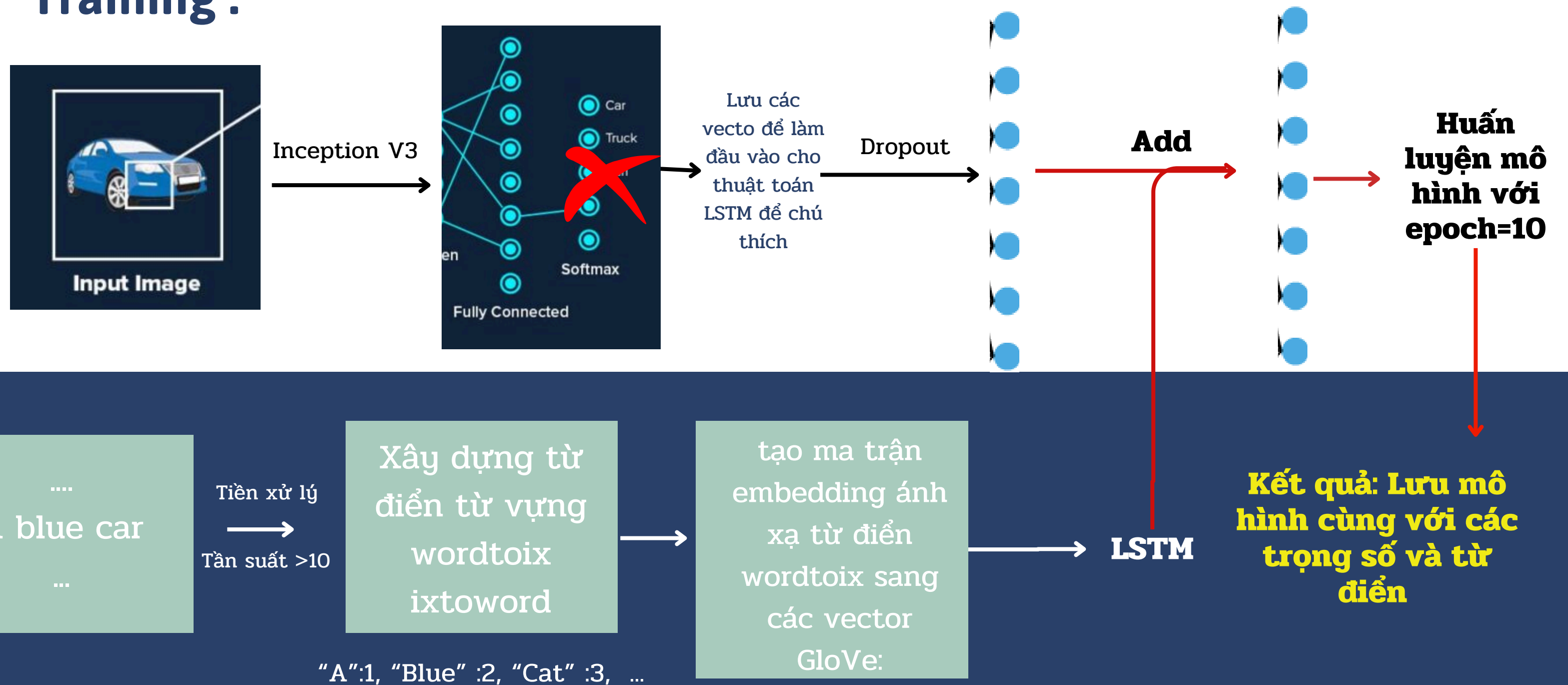


Đây là những token giúp mô hình LSTM học cách sinh câu từ các đặc trưng hình ảnh.

Sử dụng để từ hình ảnh đầu vào nhận diện và viết caption

KIẾN TRÚC

Training :



DEMO

Image Captioning Model
với
Flickr 8K và Flickr 30K



FLICKR 8K

Tác giả: Micah Hodosh, Peter Young và Julia Hockenmaier

FLICKR 30K

Tác giả: Peter Young, Alice Lai, Micah Hodosh, và Julia Hockenmaier

BỘ DỮ LIỆU SỬ DỤNG



images



captions



Mọi người, mọi vật tham gia vào các hoạt động và sự kiện hàng ngày

FLICKR 8K: 8091 ảnh

FLICKR 30K: 31783 ảnh



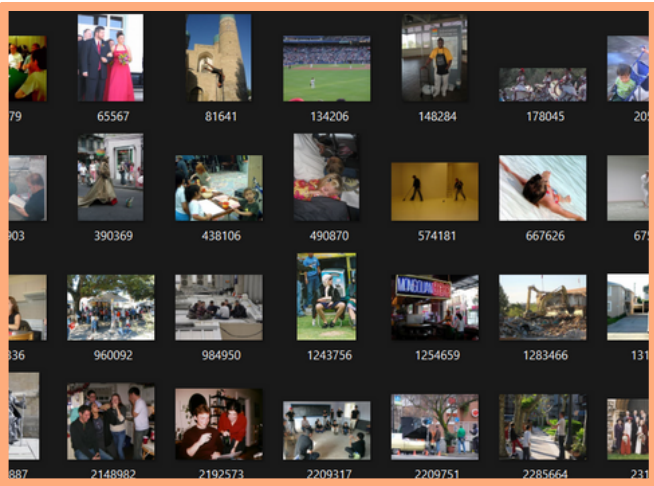
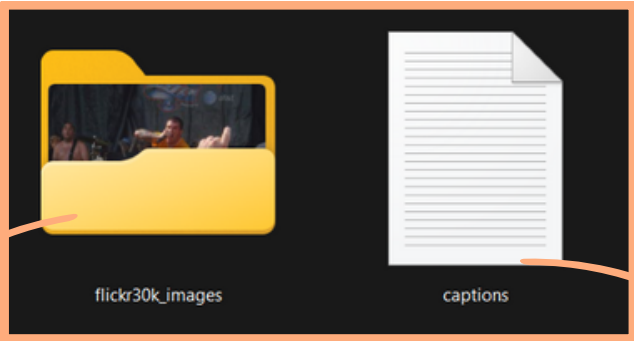
1. A climber stops to take a drink while climbing a snow covered mountain.
2. A man holding a cup on a snow mountain.
3. A man in a yellow suit is holding up a cup while standing in snow.
4. A mountain climber stops for a drink.
5. A mountaineer in a yellow jacket is drinking from a thermos cup.

BỘ DỮ LIỆU SỬ DỤNG

Bộ dữ liệu sử dụng gồm **images** (ảnh) và **captions** (mô tả) về con người, mọi vật tham gia vào các hoạt động và sự kiện hàng ngày

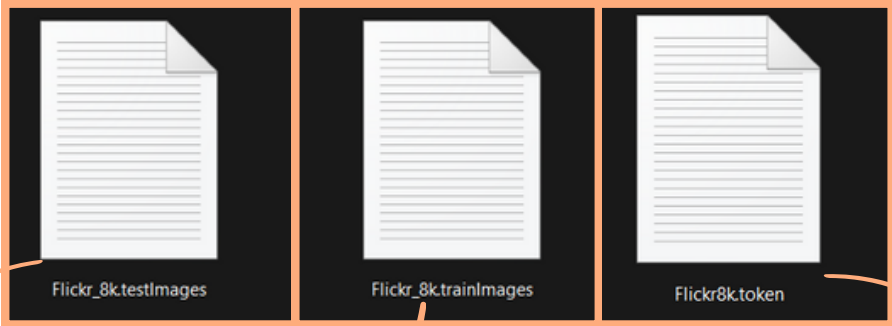


FLICKR 30K



```
image_name,comment number,comment
1000092795.jpg,0,Two young guys with shaggy hair look at their hands while
1000092795.jpg,1,Two young white males are outside near many bushes .
1000092795.jpg,2,Two men in green shirts are standing in a yard .
1000092795.jpg,3,A man in a blue shirt standing in a garden .
1000092795.jpg,4,Two friends enjoy time spent together .
10002456.jpg,0,Several men in hard hats are operating a giant pulley system
10002456.jpg,1,Workers look down from up above on a piece of equipment .
10002456.jpg,2,Two men working on a machine wearing hard hats .
10002456.jpg,3,Four men on top of a tall structure .
10002456.jpg,4,Three men on a large rig .
1000268201.jpg,0,A child in a pink dress is climbing up a set of stairs in
1000268201.jpg,1,A little girl in a pink dress going into a wooden cabin .
1000268201.jpg,2,A little girl climbing the stairs to her playhouse .
1000268201.jpg,3,A little girl climbing into a wooden playhouse
1000268201.jpg,4,A girl going into a wooden building .
1000344755.jpg,0,Someone in a blue shirt and hat is standing on stair and l
1000344755.jpg,1,A man in a blue shirt is standing on a ladder cleaning a w
1000344755.jpg,2,A man on a ladder cleans the window of a tall building .
1000344755.jpg,3,man in blue shirt and jeans on ladder cleaning windows
1000344755.jpg,4,a man on a ladder cleans a window
1000366164.jpg,0,Two men one in a gray shirt one in a black shirt standi
```

FLICKR 8K



```
3385593926_d3e9c21170.jpg
2677656448_6b7e7702af.jpg
311146855_0b65fdb169.jpg
1258913059_07c613f7ff.jpg
241347760_d44c8d3a01.jpg
2654514044_a70a6e2c21.jpg
2339106348_2df90aa6a9.jpg
256085101_2c2617c5d0.jpg
280706862_14c30d734a.jpg
3072172967_630e9c69d0.jpg
3482062809_3b694322c4.jpg
1167669558_87a8a467d6.jpg
2847615962_c330bded6e.jpg
3344233740_c010378da7.jpg
2435685480_a79d42e564.jpg
3110649716_c17e14670e.jpg
2511019188_ca71775f2d.jpg
2521770311_3086ca90de.jpg
2723477522_d89f5ac62b.jpg
2218609886_892dcd6915.jpg
3745451546_fc8ec70cbd.jpg
```

```
2513260012_03d5305cf.jpg
2903617548_d3e38d7f88.jpg
3338291921_fe7ae0c8f8.jpg
488416045_1c6d903fe0.jpg
2644326817_8f45080b87.jpg
218342358_1755a9cce1.jpg
2501968935_02f2cd8079.jpg
2699342860_5288e203ea.jpg
2638369467_8fc251595b.jpg
2926786902_815a99a154.jpg
2851304910_b5721199bc.jpg
3423802527_94bd2b23b0.jpg
3356369156_074750c6cc.jpg
2294598473_40637b5c04.jpg
1191338263_a4fa073154.jpg
2380765956_6313d8cae3.jpg
3197891333_b1b0fd1702.jpg
3119887967_271a097464.jpg
2276499757_b44dc6f8ce.jpg
```

```
1000268201_693b08cb0e.jpg#0 A child in a pink d
1000268201_693b08cb0e.jpg#1 A girl going into a
1000268201_693b08cb0e.jpg#2 A little girl climb
1000268201_693b08cb0e.jpg#3 A little girl climb
1000268201_693b08cb0e.jpg#4 A little girl in a s
1001773457_577c3a7d70.jpg#0 A black dog and a s
1001773457_577c3a7d70.jpg#1 A black dog and a t
1001773457_577c3a7d70.jpg#2 A black dog and a w
street .
1001773457_577c3a7d70.jpg#3 Two dogs of differe
1001773457_577c3a7d70.jpg#4 Two dogs on pavemen
1002674143_1b742ab4b8.jpg#0 A little girl cover
bowl .
1002674143_1b742ab4b8.jpg#1 A little girl is si
1002674143_1b742ab4b8.jpg#2 A small girl in the
rainbow on it .
1002674143_1b742ab4b8.jpg#3 There is a girl wit
1002674143_1b742ab4b8.jpg#4 Young girl with pig
1003163366_44323f5815.jpg#0 A man lays on a ben
1003163366_44323f5815.jpg#1 A man lays on the b
1003163366_44323f5815.jpg#2 a man sleeping on a
1003163366_44323f5815.jpg#3 A shirtless man lie
1003163366_44323f5815.jpg#4 man laying on bench
1007129816_e794419615.jpg#0 A man in an orange
1007129816_e794419615.jpg#1 A man in an orange
```

SƠ LƯỢC

- Xử lý tập dữ liệu Flickr 30K
- Tạo dataset hoàn chỉnh
- Tiền xử lý captions của ảnh
- Tiền xử lý ảnh
- Tạo tập từ vựng và lô dữ liệu cho huấn luyện
- Chuẩn bị ma trận cho các lớp
- Xây dựng mô hình
- Huấn luyện và lưu mô hình
- Tạo giao diện UI
- Đánh giá mô hình



XỬ LÝ TẬP DỮ LIỆU FLICKR 30K

Caption của Flickr 8K

```
1000268201_693b08cb0e.jpg#0      A child in a pink dress is cl
1000268201_693b08cb0e.jpg#1      A girl going into a wooden bu
1000268201_693b08cb0e.jpg#2      A little girl climbing into a
```

Caption của Flickr 30K

```
image_name,comment_number,comment
1000092795.jpg,0,Two young guys with shaggy hair look at the
1000092795.jpg,1,Two young White males are outside near mar
1000092795.jpg,2,Two men in green shirts are standing in a
```

Đọc và
chỉnh sửa
captions
của
Flickr 30K

```
import pandas as pd

# Đọc file captions.txt của Flickr30k
captions_file = r'C:\Users\ADMIN\Downloads\Train model\captions.txt'
captions_data = pd.read_csv(captions_file,
                             names=['image_name', 'comment_number', 'comment'])

# Chuyển đổi cấu trúc dữ liệu để phù hợp với Flickr8k
output_file = r'C:\Users\ADMIN\Downloads\Train model\Flickr30k.token.txt'
with open(output_file, 'w') as f:
    for _, row in captions_data.iterrows():
        f.write(f"{row['image_name']}#{row['comment_number']}\t{row['comment']}\n")
```


TẠO DATASET HOÀN CHỈNH

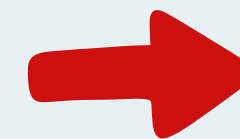
```
import os
import random

# Đường dẫn thư mục chứa ảnh
images_dir = r'C:\Users\ADMIN\Downloads\Train model\flickr30k_images'
all_images = os.listdir(images_dir)

# Chia dữ liệu train/test (80% train, 20% test)
random.shuffle(all_images)
train_split = int(0.8 * len(all_images))
train_images = all_images[:train_split]
test_images = all_images[train_split:]

# Lưu danh sách ảnh vào file
with open('Flickr30k.trainImages.txt', 'w') as f:
    for img in train_images:
        f.write(img + '\n')

with open('Flickr30k.testImages.txt', 'w') as f:
    for img in test_images:
        f.write(img + '\n')
```



Train: 25426 ảnh
Test: 6357 ảnh

Chia dữ liệu thành tập huấn luyện và kiểm tra

TẠO DATASET HOÀN CHỈNH

```
test_images_1 = r"C:\Users\ADMIN\Downloads\Train model\Flickr_8k.testImages.txt"
test_images_2 = r"C:\Users\ADMIN\Downloads\Train model\Flickr30k.testImages.txt"
output_test_file = r"C:\Users\ADMIN\Downloads\Train model\Test_Images.txt"

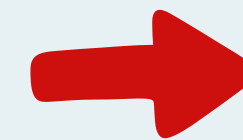
train_images_1 = r"C:\Users\ADMIN\Downloads\Train model\Flickr30k.trainImages.txt"
train_images_2 = r"C:\Users\ADMIN\Downloads\Train model\Flickr_8k.trainImages.txt"
output_train_file = r"C:\Users\ADMIN\Downloads\Train model\Train_Images.txt"

def merge_files(file1, file2, output_file):
    with open(output_file, 'w', encoding='utf-8') as outfile:
        # Ghi nội dung từ file1
        with open(file1, 'r', encoding='utf-8') as infile1:
            outfile.write(infile1.read())
        # Ghi nội dung từ file2
        with open(file2, 'r', encoding='utf-8') as infile2:
            outfile.write(infile2.read())

# Gộp các file test
merge_files(test_images_1, test_images_2, output_test_file)

# Gộp các file train
merge_files(train_images_1, train_images_2, output_train_file)

print("Gộp file hoàn tất!")
```



Train: 31426 ảnh
Test: 7357 ảnh

Gộp file huấn luyện và test của hai bộ dữ liệu

Hải Anh

TẠO DATASET HOÀN CHỈNH

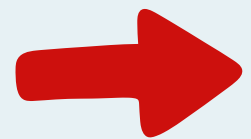
```
token_1 = r"C:\Users\ADMIN\Downloads\Train model\Flickr8k.token.txt"
token_2 = r"C:\Users\ADMIN\Downloads\Train model\Flickr30k.token.txt"

from charset_normalizer import detect

def detect_file_encoding(file_path):
    with open(file_path, 'rb') as f:
        raw_data = f.read()
        result = detect(raw_data)
        return result['encoding']

# Kiểm tra mã hóa
encoding1 = detect_file_encoding(token_1)
encoding2 = detect_file_encoding(token_2)

print(f"Mã hóa của {token_1}: {encoding1}")
print(f"Mã hóa của {token_2}: {encoding2}")
```



Mã hóa của [C:\Users\ADMIN\Downloads\Train model\Flickr8k.token.txt](#): utf-8
Mã hóa của [C:\Users\ADMIN\Downloads\Train model\Flickr30k.token.txt](#): ascii

Xác định loại mã hoá sử dụng trong 2 file caption

TẠO DATASET HOÀN CHỈNH

```
def convert_to_utf8(input_file, output_file, source_encoding):  
    with open(input_file, 'r', encoding=source_encoding, errors='replace') as infile,  
        open(output_file, 'w', encoding='utf-8') as outfile:  
        outfile.write(infile.read())
```

Đường dẫn tới file

```
token_2 = r"C:\Users\ADMIN\Downloads\Train model\Flickr30k.token.txt" # windows-1250
```

```
converted_token_2 = r"C:\Users\ADMIN\Downloads\Train model\Flickr30k_con.token.txt"
```

Chuyển đổi từ windows-1250 sang utf-8

```
convert_to_utf8(token_2, converted_token_2, 'ascii')
```

```
print(f"Chuyển đổi {token_2} sang UTF-8 hoàn tất!")
```

Chuyển đổi mã hóa sang UTF-8

```
output_token_file = r"C:\Users\ADMIN\Downloads\Train model\Images.token.txt"
```

```
merge_files(token_1, converted_token_2, output_token_file)
```

Gộp 2 file captions

IMPORT THƯ VIỆN

```
# Thêm thư viện
import numpy as np
import string
import glob
from pickle import load
from time import time

# Sử dụng tensorflow.keras thay vì keras
from tensorflow.keras.models import Model
from tensorflow.keras.layers import LSTM, Embedding, Dense, Dropout, add, Input
from tensorflow.keras.applications.inception_v3 import InceptionV3, preprocess_input
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
```


TIỀN XỬ LÝ CAPTIONS CỦA ẢNH

```
# Load the caption file
def load_doc(filename):
    # open the file as read only
    file = open(filename, 'r')
    # read all text
    text = file.read()
    # close the file
    file.close()
    return text

filename = "C:/Users/ADMIN/Downloads/Train model/Images.token.txt"

doc = load_doc(filename)
print(doc[:300])
```



1000268201_693b08cb0e.jpg#0	A child in a pink dress is climbing up a set of stairs in an entry way .
1000268201_693b08cb0e.jpg#1	A girl going into a wooden building .
1000268201_693b08cb0e.jpg#2	A little girl climbing into a wooden playhouse .
1000268201_693b08cb0e.jpg#3	A little girl climbing the s

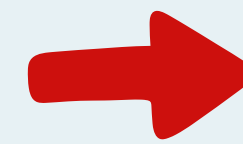
Hàm để tải nội dung của file và hiển thị 300 chữ đầu

TIỀN XỬ LÝ CAPTIONS CỦA ẢNH

```
def load_descriptions(doc):
    mapping = dict()
    # process lines
    for line in doc.split('\n'):
        # split line by white space
        tokens = line.split()
        if len(line) < 2:
            continue
        # take the first token as the image id, the rest as the
        image_id, image_desc = tokens[0], tokens[1:]
        # extract filename from image id
        image_id = image_id.split('.')[0]
        # convert description tokens back to string
        image_desc = ' '.join(image_desc)
        # create the list if needed
        if image_id not in mapping:
            mapping[image_id] = list()
        # store description
        mapping[image_id].append(image_desc)
    return mapping
```

```
descriptions = load_descriptions(doc)
print('Loaded: %d ' % len(descriptions))
descriptions['1000268201_693b08cb0e']
```

Tách caption và lưu dưới dạng cặp **khóa-giá trị** với tên hình ảnh là khóa và mô tả là giá trị.



Loaded: 39875

```
['A child in a pink dress is climbing up a set of stairs in an
'A girl going into a wooden building .',
'A little girl climbing into a wooden playhouse .',
'A little girl climbing the stairs to her playhouse .',
'A little girl in a pink dress going into a wooden cabin .']
```

TIỀN XỬ LÝ CAPTIONS CỦA ẢNH

```
def clean_descriptions(descriptions):  
    # prepare translation table for removing punctuation  
    table = str.maketrans('', '', string.punctuation)  
    for key, desc_list in descriptions.items():  
        for i in range(len(desc_list)):  
            desc = desc_list[i]  
            # tokenize  
            desc = desc.split()  
            # convert to lower case  
            desc = [word.lower() for word in desc]  
            # remove punctuation from each token  
            desc = [w.translate(table) for w in desc]  
            # remove hanging 's' and 'a'  
            desc = [word for word in desc if len(word) > 1]  
            # remove tokens with numbers in them  
            desc = [word for word in desc if word.isalpha()]  
            # store as string  
            desc_list[i] = ' '.join(desc)  
  
# clean descriptions  
clean_descriptions(descriptions)  
descriptions['1000268201_693b08cb0e']
```

Hàm tiền xử lý các caption bằng cách loại bỏ dấu câu, chuyển chữ hoa thành chữ thường, và loại bỏ các từ không cần thiết.

```
['child in pink dress is climbing up set of stairs in  
'girl going into wooden building',  
'little girl climbing into wooden playhouse',  
'little girl climbing the stairs to her playhouse',  
'little girl in pink dress going into wooden cabin']
```

Kết quả

TIỀN XỬ LÝ CAPTIONS CỦA ẢNH

```
# Save descriptions to file
def save_descriptions(descriptions, filename):
    lines = list()
    for key, desc_list in descriptions.items():
        for desc in desc_list:
            lines.append(key + ' ' + desc)
    data = '\n'.join(lines)
    file = open(filename, 'w')
    file.write(data)
    file.close()

save_descriptions(descriptions, 'descriptions.txt')
```

Lưu captions của ảnh vào file descriptions.txt

TIỀN XỬ LÝ CAPTIONS CỦA ẢNH

```
def load_clean_descriptions(filename, dataset):  
    # Load document  
    doc = load_doc(filename)  
    descriptions = dict()  
    for line in doc.split('\n'):  
        # split line by white space  
        tokens = line.split()  
        # split id from description  
        image_id, image_desc = tokens[0], tokens[1:]  
        # skip images not in the set  
        if image_id in dataset:  
            # create list  
            if image_id not in descriptions:  
                descriptions[image_id] = list()  
            # wrap description in tokens  
            desc = 'startseq ' + ' '.join(image_desc) + ' endseq'  
            # store  
            descriptions[image_id].append(desc)  
    return descriptions  
  
# descriptions  
train_descriptions = load_clean_descriptions('descriptions.txt', train)  
print('Descriptions: train=%d' % len(train_descriptions))
```

Hàm để thêm các từ
'startseq' và 'endseq' vào
mỗi caption để đánh dấu
bắt đầu và kết thúc của câu.

Descriptions: train=31426

Kết quả

TIỀN XỬ LÝ ẢNH

```
images = r'C:\Users\ADMIN\Downloads\Train model\Dataset Images/'  
# Get all jpg images in the folder  
img = glob.glob(images + '*.jpg')
```

Lấy tất cả các đường dẫn của tệp đuôi .jpg trong thư mục chỉ định.

```
train_images_file = r'C:\Users\ADMIN\Downloads\Train model\Train_Images.txt'  
# Read the train image names in a set  
train_images = set(open(train_images_file, 'r').read().strip().split('\n'))  
  
# Create a list of all the training images with their full path names  
train_img = []  
  
for i in img: # img is list of full path names of all images  
    if i[len(images):] in train_images: # Check if the image belongs to tra  
        train_img.append(i) # Add it to the list of train images
```

Tạo danh sách các ảnh có đầy đủ đường dẫn để dùng cho Train / Test.

Hải Anh

TIỀN XỬ LÝ ẢNH

```
def preprocess(image_path):  
    # Convert all the images to size 299x299 as expected by  
    img = image.load_img(image_path, target_size=(299, 299))  
    # Convert PIL image to numpy array of 3-dimensions  
    x = image.img_to_array(img)  
    # Add one more dimension  
    x = np.expand_dims(x, axis=0)  
    # preprocess the images using preprocess_input() from inc  
    x = preprocess_input(x)  
    return x
```

```
model = InceptionV3(weights='imagenet')  
# Create a new model, removing the last layer from incep  
model_new = Model(model.input, model.layers[-2].output)
```

```
def encode(image):  
    image = preprocess(image) # preprocess the image  
    fea_vec = model_new.predict(image) # Get the en  
    fea_vec = np.reshape(fea_vec, fea_vec.shape[1])  
    return fea_vec
```

Tiền xử lý ảnh để phù hợp với yêu cầu của mô hình InceptionV3.

Tạo một mô hình mới bằng cách loại bỏ lớp cuối cùng của InceptionV3.

Mã hóa ảnh thành một vector đặc trưng sử dụng mô hình InceptionV3.

TIỀN XỬ LÝ ẢNH

```
start = time()
encoding_train = {}
for img in train_img:
    encoding_train[img[len(images):]] = encode(img)
print("Time taken in seconds =", time() - start)
```

Mã hoá ảnh trong tập
Train và Test thành một
vector đặc trưng

```
1/1 _____ 0s 58ms/step
1/1 _____ 0s 58ms/step
1/1 _____ 0s 60ms/step
...
1/1 _____ 0s 123ms/step
1/1 _____ 0s 95ms/step
1/1 _____ 0s 90ms/step
Time taken in seconds = 4439.038126707
```

```
1/1 _____ 0s 77ms/step
1/1 _____ 0s 85ms/step
1/1 _____ 0s 85ms/step
...
1/1 _____ 0s 72ms/step
1/1 _____ 0s 76ms/step
1/1 _____ 0s 88ms/step
Time taken in seconds = 1055.29702377
```

Thời gian mã hoá và lưu kết quả vào file

 encoded_train_images.pkl	06/12/2024 10:07 SA
 encoded_test_images.pkl	06/12/2024 10:25 SA

TẠO TẬP TỪ VỰNG

```
all_train_captions = []
for key, val in train_descriptions.items():
    for cap in val:
        all_train_captions.append(cap)
print('Total train captions: %d' % len(all_train_captions))
```

Total train captions: 157130

Tạo danh sách chứa tất cả các captions và in ra độ dài của danh sách

```
word_count_threshold = 10
word_counts = {}
nsents = 0
for sent in all_train_captions:
    nsents += 1
    for w in sent.split(' '):
        word_counts[w] = word_counts.get(w, 0) + 1

vocab = [w for w in word_counts if word_counts[w] >= word_count_threshold]
print('Preprocessed words %d -> %d' % (len(word_counts), len(vocab)))
```

Preprocessed words 18341 -> 5194

Tạo tập từ vựng chỉ chứa những từ xuất hiện ít nhất 10 lần trong tập huấn luyện.

TẠO TẬP TỪ VỰNG

```
ixtoword = {}  
wordtoix = {}  
ix = 1  
for w in vocab:  
    wordtoix[w] = ix  
    ixtoword[ix] = w  
    ix += 1  
  
vocab_size = len(ixtoword) + 1 # one  
print('Vocabulary size:', vocab_size)
```

Kết quả

Vocabulary size: 5195

Tạo ánh xạ từ (word)
sang chỉ mục (index)
và ngược lại để sử
dụng trong mô hình.

```
max_length = max(len(d.split())) for d in all_train_captions)  
print('Description Length: %d' % max_length)
```

Xác định độ dài tối đa của một mô tả để dùng trong padding.

Kết quả

Description Length: 70

TẠO LÔ DỮ LIỆU CHO HUẤN LUYỆN

```
def data_generator(descriptions, photos, wordtoix, max_length, num_photos_per_batch):  
    while True:  
        X1, X2, y = list(), list(), list()  
        n = 0  
        for key, desc_list in descriptions.items():  
            n += 1  
            photo = photos[key + '.jpg']  
            for desc in desc_list:  
                seq = [wordtoix[word] for word in desc.split(' ') if word in wordtoix]  
                for i in range(1, len(seq)):  
                    in_seq, out_seq = seq[:i], seq[i]  
                    in_seq = pad_sequences([in_seq], maxlen=max_length)[0]  
                    out_seq = to_categorical([out_seq], num_classes=vocab_size)[0]  
                    X1.append(photo)  
                    X2.append(in_seq)  
                    y.append(out_seq)  
            if n == num_photos_per_batch:  
                yield (np.array(X1), np.array(X2)), np.array(y)  
                X1, X2, y = list(), list(), list()  
                n = 0
```

Hàm sinh dữ liệu theo lô để huấn luyện mô hình.

CHUẨN BỊ MA TRẬN CHO CÁC LỚP

```
glove_dir = r'C:\Users\ADMIN\Downloads\Train model\glove.6B'

embeddings_index = {}

f = open(os.path.join(glove_dir, 'glove.6B.200d.txt'), encoding='utf-8')

for line in f:
    values = line.split()
    word = values[0]
    coefs = np.asarray(values[1:], dtype='float32')
    embeddings_index[word] = coefs
f.close()

print('Found %s word vectors.' % len(embeddings_index))
```

Đọc file chứa các vector từ GloVe để sử dụng trong việc tạo ma trận embedding.

Kết quả Found 400001 word vectors.

CHUẨN BỊ MA TRẬN CHO CÁC LỚP

Tạo ma trận embedding để sử dụng trong lớp embedding của mô hình.

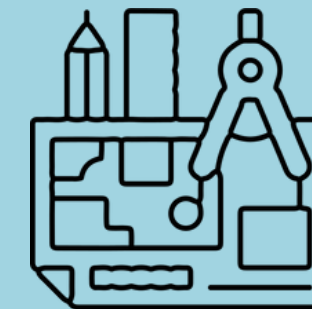
```
embedding_matrix = np.zeros((vocab_size, 200))
for word, i in wordtoix.items():
    if i >= vocab_size:
        continue
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is not None:
        embedding_matrix[i] = embedding_vector
embedding_matrix.shape
```

Kết quả

(5195, 200)



THIẾT KẾ MÔ HÌNH



```
embedding_layer = Embedding(vocab_size, 200, weights=[embedding_matrix],
input_length=max_length, trainable=False)

inputs1 = Input(shape=(2048,))
fe1 = Dropout(0.5)(inputs1)
fe2 = Dense(256, activation='relu')(fe1)

inputs2 = Input(shape=(max_length,))
se1 = embedding_layer(inputs2)
se2 = LSTM(256)(se1)

decoder1 = add([fe2, se2])
decoder2 = Dense(256, activation='relu')(decoder1)
outputs = Dense(vocab_size, activation='softmax')(decoder2)

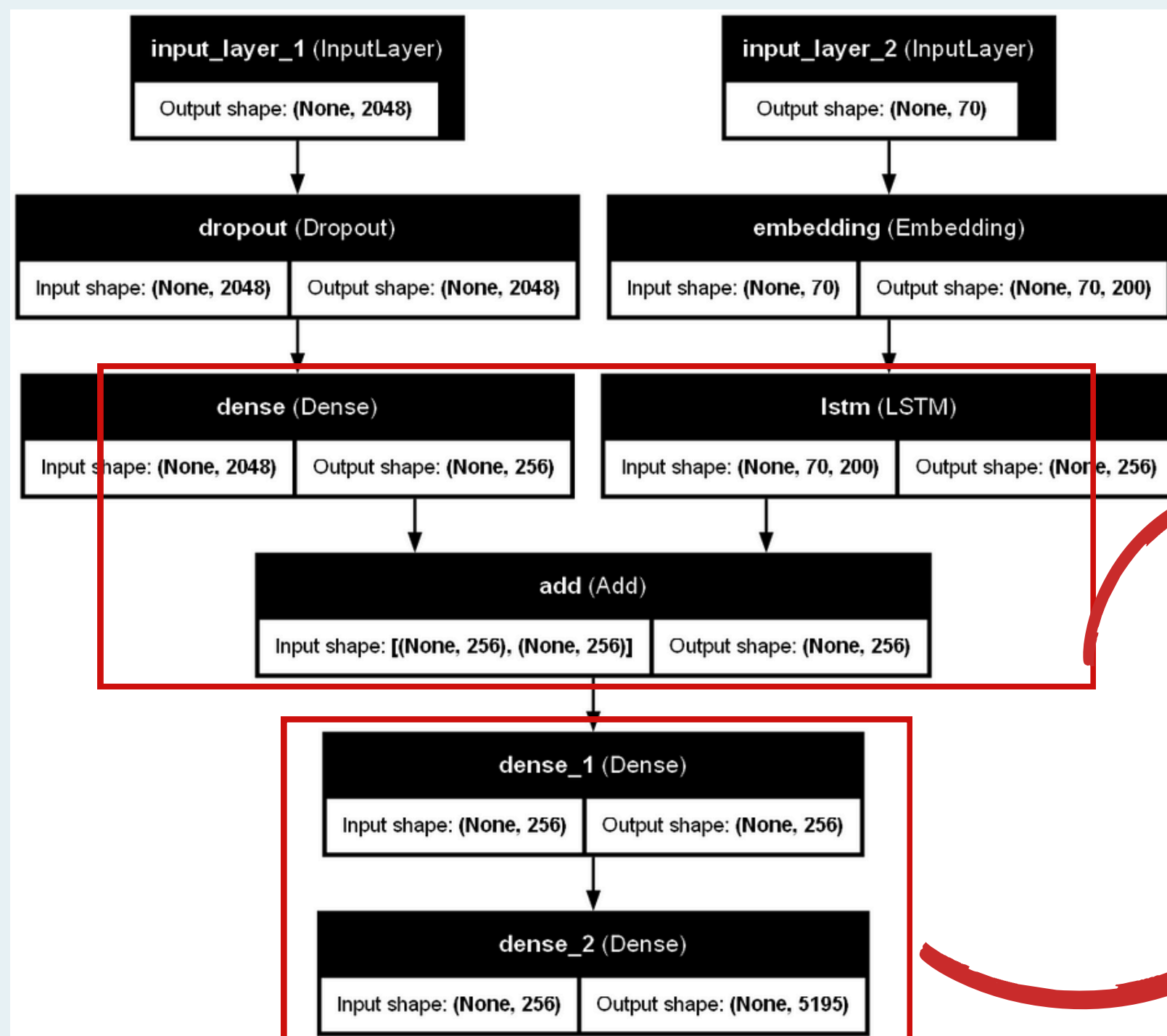
model = Model(inputs=[inputs1, inputs2], outputs=outputs)
model.compile(loss='categorical_crossentropy', optimizer='adam')

model.summary()
```

- Input Layers: Nhận dữ liệu đầu vào (một vector đặc trưng và chuỗi).
- Dropout: Giảm overfitting bằng cách ngẫu nhiên bỏ qua các kết nối thần kinh.
- Dense: Giảm chiều thông qua ma trận trọng số và hàm kích hoạt



THIẾT KẾ MÔ HÌNH

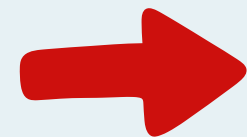


- Embedding: Chuyển đổi các từ thành vector số học có kích thước cố định (200).
- LSTM: Tạo vector ngữ cảnh cho từ.
- Add: Tổng hợp kết quả từ nhiều lớp.
- Final Dense Layer: Dự đoán đầu ra cuối cùng

CÀI ĐẶT THÔNG SỐ HUẤN LUYỆN VÀ CHUẨN BỊ DỮ LIỆU

```
model.optimizer.lr = 0.0001
epochs = 10
number_pics_per_batch = 32
steps = len(train_descriptions) // number_pics_per_batch
```

```
for i in range(epochs):
    generator = data_generator(train_descriptions, train_features, wordtoix, max_length, number_pics_per_batch)
    model.fit(generator, epochs=1, steps_per_epoch=steps, verbose=1)
```



```
c:\Users\ADMIN\AppData\Local\Programs\Python\Python312\Lib\site-
warnings.warn(
982/982 _____ 2023s 2s/step - loss: 4.8158
982/982 _____ 2061s 2s/step - loss: 3.4682
982/982 _____ 2125s 2s/step - loss: 3.2254
982/982 _____ 2243s 2s/step - loss: 3.0826
982/982 _____ 2251s 2s/step - loss: 2.9813
982/982 _____ 2346s 2s/step - loss: 2.9014
982/982 _____ 2412s 2s/step - loss: 2.8352
982/982 _____ 2637s 3s/step - loss: 2.7793
982/982 _____ 2747s 3s/step - loss: 2.7313
982/982 _____ 2861s 3s/step - loss: 2.6880
```

Cài đặt các thông số quan trọng cho quá trình huấn luyện -> ảnh hưởng trực tiếp đến tốc độ và hiệu quả tối ưu của mô hình



Phương Anh

CẤU HÌNH VÀ LƯU TRỮ MÔ HÌNH

```
from keras.optimizers import Adam
```

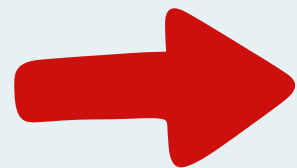
```
# Biên dịch lại mô hình
```

```
model.compile(optimizer=Adam(learning_rate=0.0001),  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

Bộ tối ưu hóa Adam với tốc độ học nhỏ (0.0001), hàm mất mát categorical crossentropy

```
# Save the entire model (architecture + weights + optimizer state)
```

```
model.save('C:/Users/ADMIN/Downloads/train model/model_new.keras', include_optimizer=True)
```



Đảm bảo mô hình được tối ưu hóa đúng cách và có thể tái sử dụng sau khi huấn luyện



Phương Anh

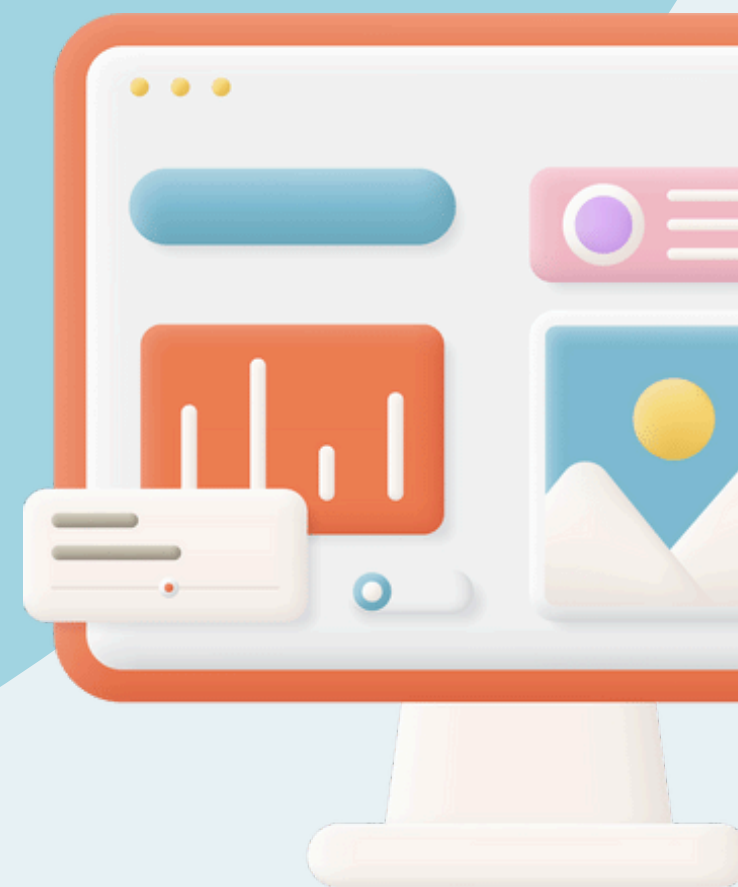
- UI -

GIAO DIỆN

NGƯỜI DÙNG

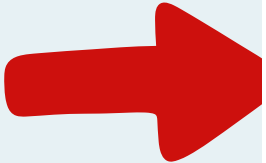


Phương Anh



GIAO DIỆN NGƯỜI DÙNG

```
# Tạo giao diện Gradio
iface = gr.Interface(fn=predict_caption, inputs=gr.Image(type="filepath"), outputs=gr.Text())
iface.launch()
share=True
```

- 
1. Sử dụng **mô hình đã được huấn luyện** để dự đoán chú thích từ ảnh.
 2. **Tiền xử lý ảnh**: Tất cả ảnh đầu vào được resize và chuẩn hóa trước khi đưa vào mô hình mã hóa.
 3. **Mã hóa ảnh**: InceptionV3 được dùng để trích xuất vector đặc trưng 2048 chiều từ ảnh.
 4. **Sinh chú thích**: Vector đặc trưng và mô hình Image Captioning kết hợp để dự đoán từng từ của câu.
 5. **Giao diện Gradio**: giao diện đơn giản, trực quan, cho phép người dùng tải ảnh lên và nhận chú thích tương ứng.

ĐÁNH GIÁ MÔ HÌNH



Phương Anh





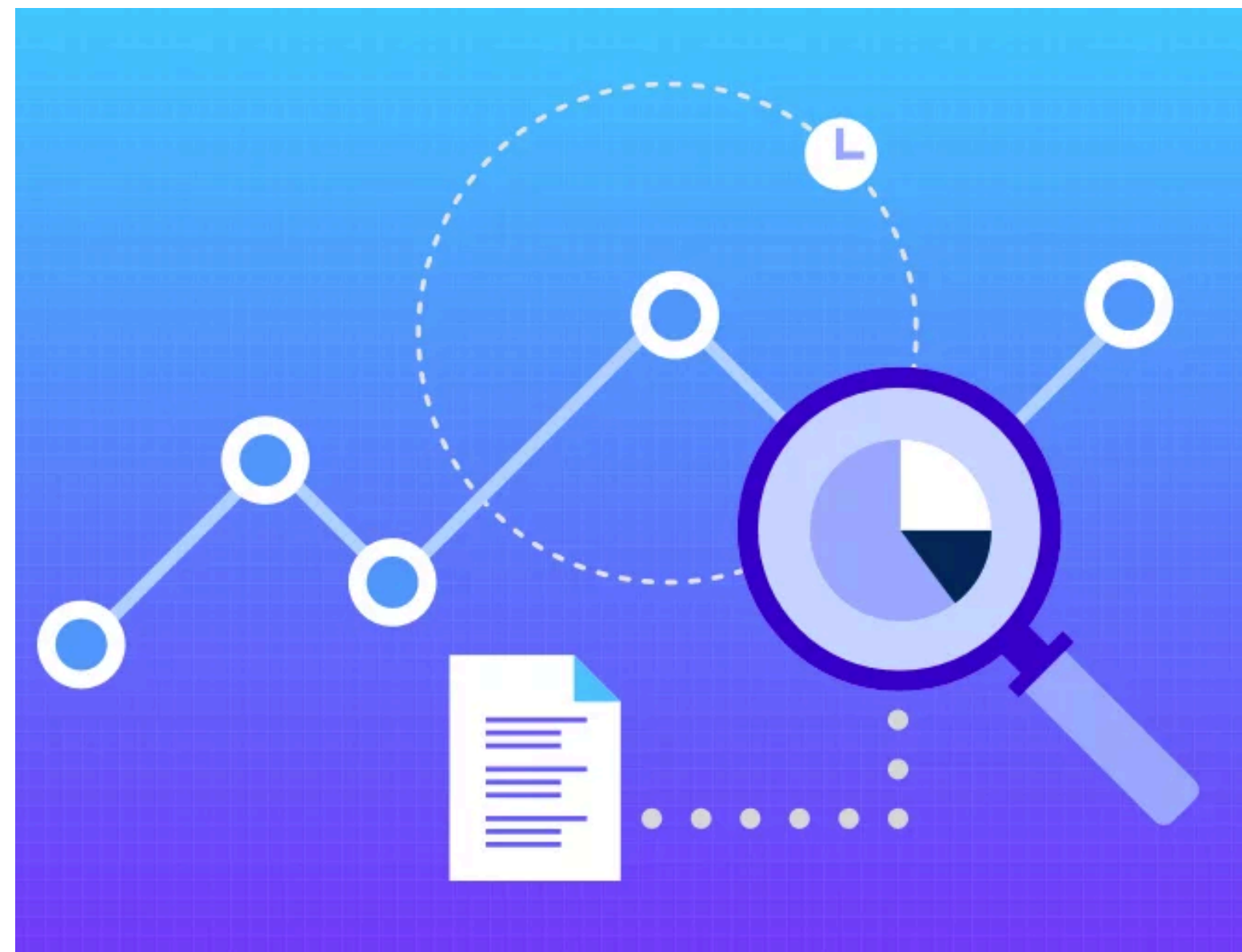
CHỈ SỐ ĐÁNH GIÁ BLEU



BLEU (Bilingual Evaluation Understudy) là một trong những chỉ số phổ biến nhất để đánh giá chất lượng các mô hình dịch máy hoặc các hệ thống tạo văn bản tự động.

BLEU đánh giá chất lượng của một văn bản dựa trên tần suất xuất hiện của các cụm từ (n-grams) trong văn bản mô hình tạo ra so với văn bản tham chiếu.

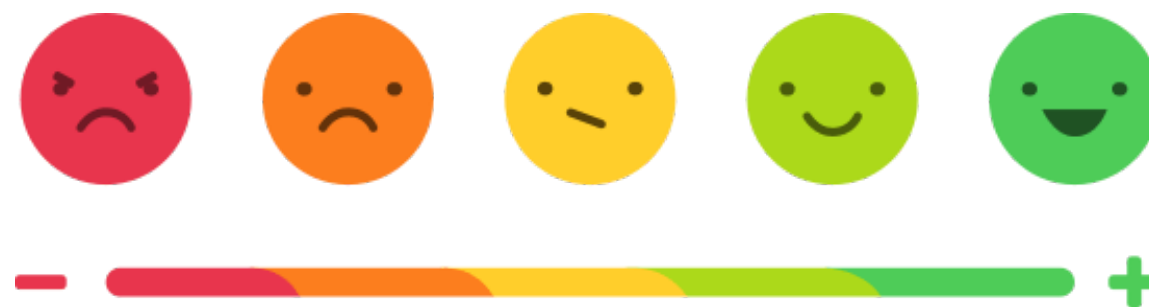
Chú ý các điểm: BLEU-1, BLEU-2, BLEU-3, BLEU-N



CHỈ SỐ ĐÁNH GIÁ BLEU

Tổng kết

- BLEU so sánh văn bản dự đoán với tất cả các tham chiếu -> chọn kết quả tốt nhất.
 - BLEU không xét đến ngữ nghĩa, chỉ tập trung vào tần suất từ hoặc n-gram.
- => Ý nghĩa tương tự nhưng khác biệt về cấu trúc có thể nhận điểm BLEU thấp.
- Bleu Score = [0,1]. Mức độ 0.6 hoặc 0.7 được coi như là tốt.



CODE ĐÁNH GIÁ MÔ HÌNH

```
# Hàm tính điểm BLEU
def evaluate_model_bleu(model, descriptions, photos, wordtoix, ixtoword, max_length):
    smooth = SmoothingFunction().method4
    actual, predicted = list(), list()
    for key, desc_list in descriptions.items():
        y_pred = generate_caption(model, photos[key + '.jpg'].reshape((1, 2048)), wordtoix, ixtoword, max_length)
        references = [desc.split() for desc in desc_list]
        actual.append(references)
        predicted.append(y_pred)

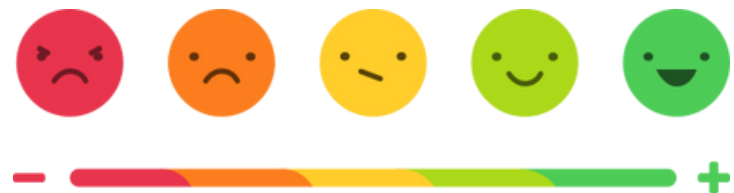
    # Tính BLEU-1, BLEU-2, BLEU-3, BLEU-4
    bleu_scores = {"BLEU-1": [], "BLEU-2": [], "BLEU-3": [], "BLEU-4": []}
    for ref, pred in zip(actual, predicted):
        bleu_scores["BLEU-1"].append(sentence_bleu(ref, pred, weights=(1, 0, 0, 0), smoothing_function=smooth))
        bleu_scores["BLEU-2"].append(sentence_bleu(ref, pred, weights=(0.5, 0.5, 0, 0), smoothing_function=smooth))
        bleu_scores["BLEU-3"].append(sentence_bleu(ref, pred, weights=(0.33, 0.33, 0.33, 0), smoothing_function=smooth))
        bleu_scores["BLEU-4"].append(sentence_bleu(ref, pred, weights=(0.25, 0.25, 0.25, 0.25), smoothing_function=smooth))

    # Trung bình điểm BLEU cho toàn bộ tập
    for key in bleu_scores:
        bleu_scores[key] = np.mean(bleu_scores[key])
    return bleu_scores
```

So sánh câu dự đoán
với câu tham chiếu.

Tính điểm BLEU
cho các cấp độ từ
unigram -> 4-gram

Trung bình điểm
BLEU trên toàn bộ
tập kiểm tra





KẾT QUẢ

```
for k, v in dict.items():  
    print(f"{k}:  
[65]  
... BLEU Scores:  
BLEU-1: 0.4426  
BLEU-2: 0.2540  
BLEU-3: 0.1574  
BLEU-4: 0.1009
```

- **BLEU-1 (Unigram precision):**

- Điểm số BLEU-1 khá cao (44.26%) -> các từ đơn lẻ trong văn bản dự đoán khớp tương đối tốt với văn bản tham chiếu.
=> Từ vựng trong dự đoán được lựa chọn phù hợp và có liên quan đến văn bản tham chiếu.

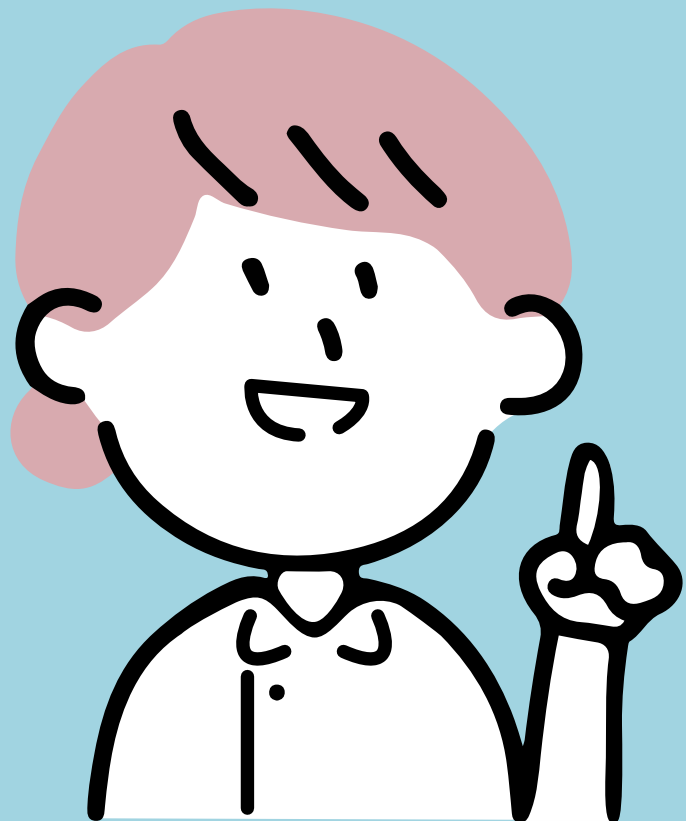
- **BLEU-2 (Bigram precision):**

- BLEU-2 giảm xuống còn 25.40% -> một số bigram (cặp từ liên tiếp) không khớp hoàn toàn với tham chiếu.
- Mô hình gặp khó khăn trong việc tạo ra các cụm từ hoặc cấu trúc câu dài chính xác.



Ý TƯỞNG MỞ RỘNG

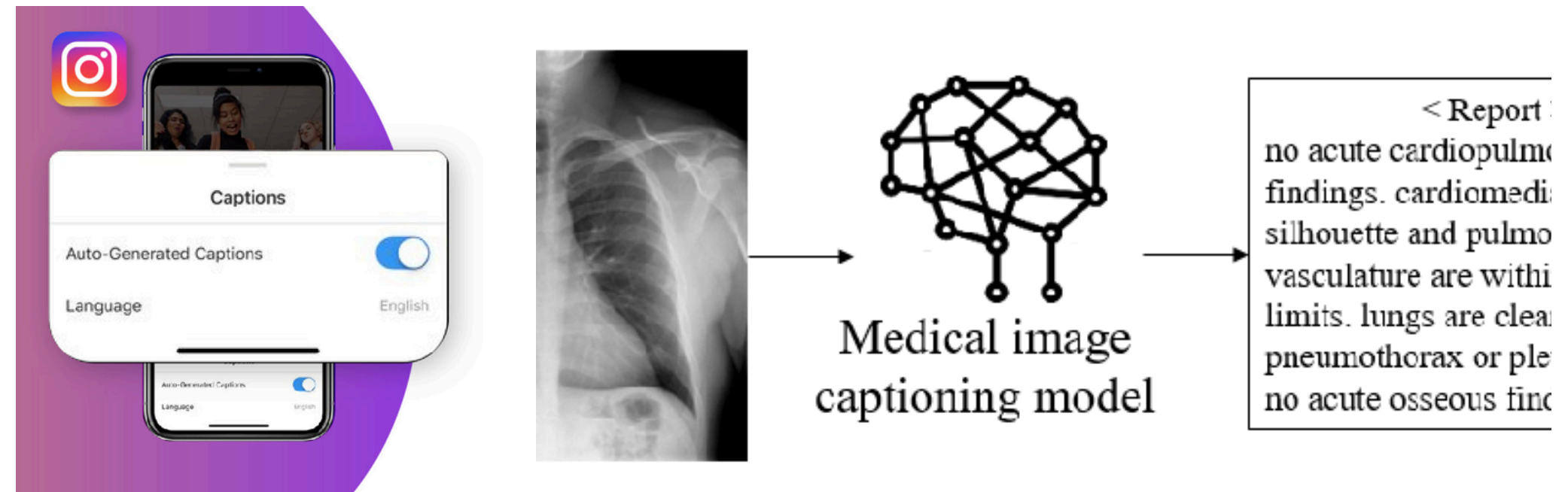
- **Tăng dữ liệu đầu vào:** Việc tăng số lượng ảnh đầu vào giúp mô hình có thêm nhiều dữ liệu để học, mở rộng mô hình, tăng khả năng ứng dụng vào thực tế
- **Tối ưu hoá mô hình:** Có thể khám phá cấu hình, thuật toán khác để đạt được hiệu suất tốt hơn
- **Sử dụng nhiều chỉ số đánh giá:** Để đánh giá chính xác trên nhiều khía cạnh khác của mô hình hơn chúng ta có thể sử dụng nhiều chỉ số khác như **METEOR**, **ROUGE**, hoặc **TER**
- **Mở rộng Real-time:** Không chỉ dừng lại ở việc nhận diện hình ảnh, chúng ta có thể mở rộng thêm nhận diện hình ảnh với real-time



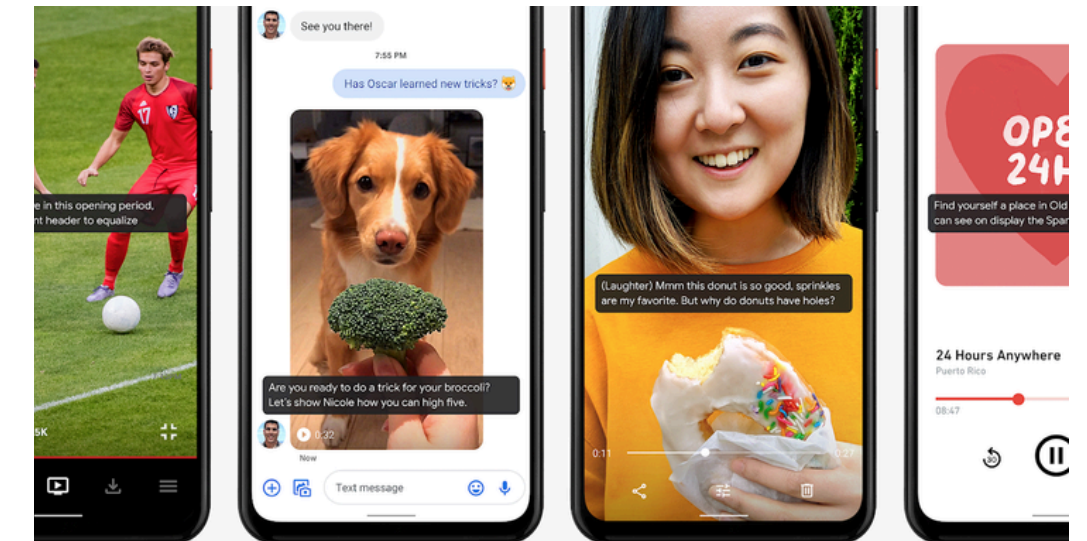
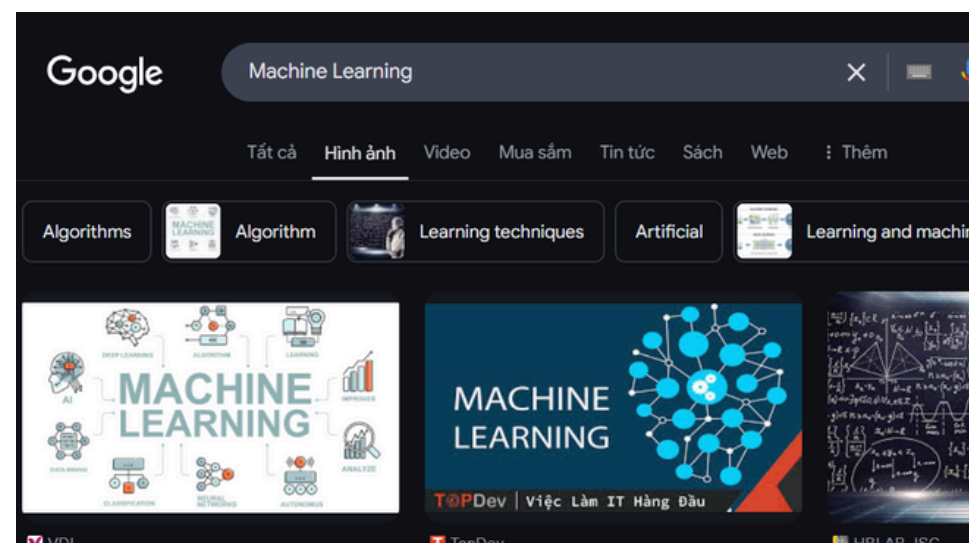
ỨNG DỤNG THỰC TẾ

Image Captioning
(Mô tả ảnh)

Hỗ trợ người khiếm thị



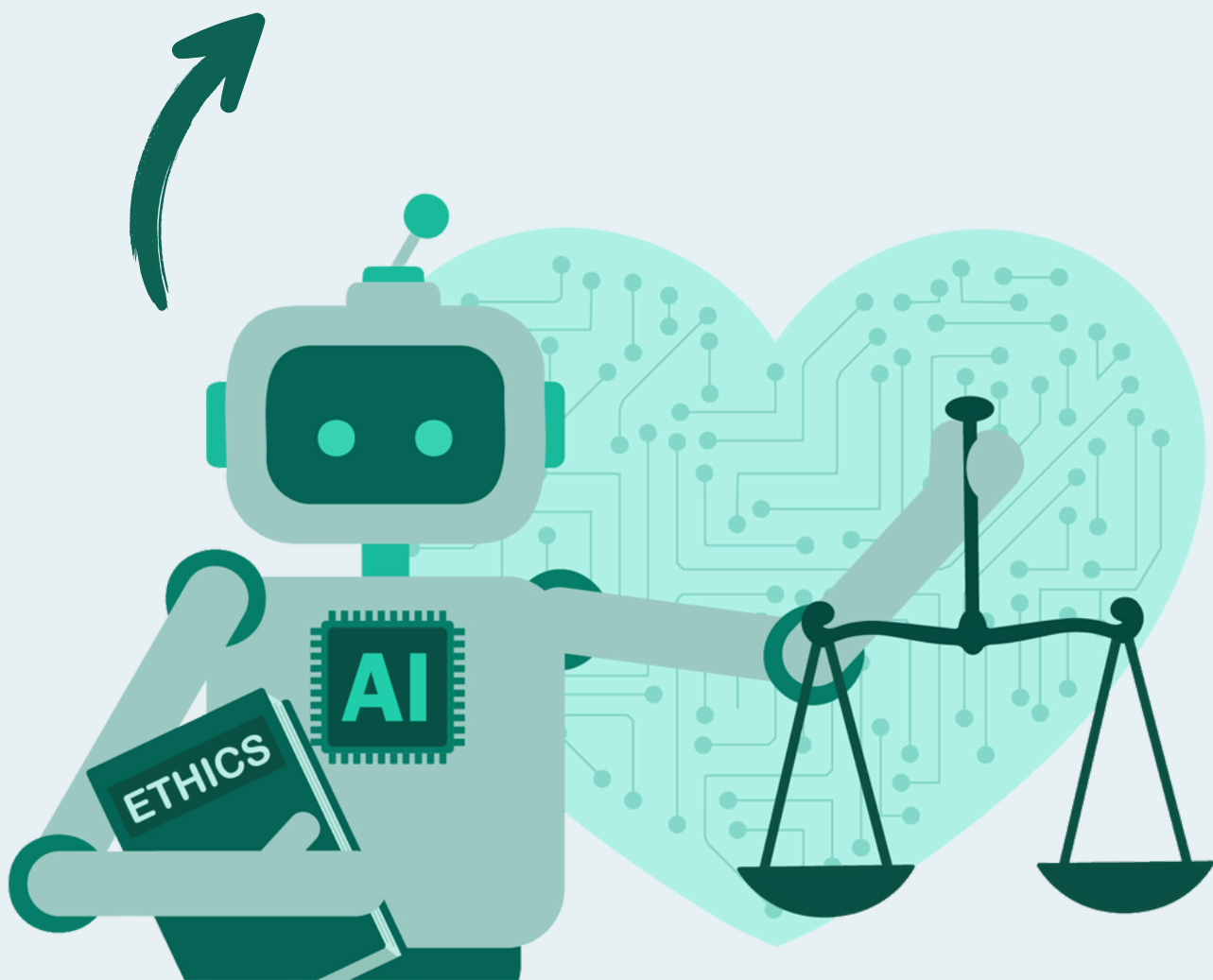
Google tìm kiếm



ĐẠO ĐỨC TRONG AI

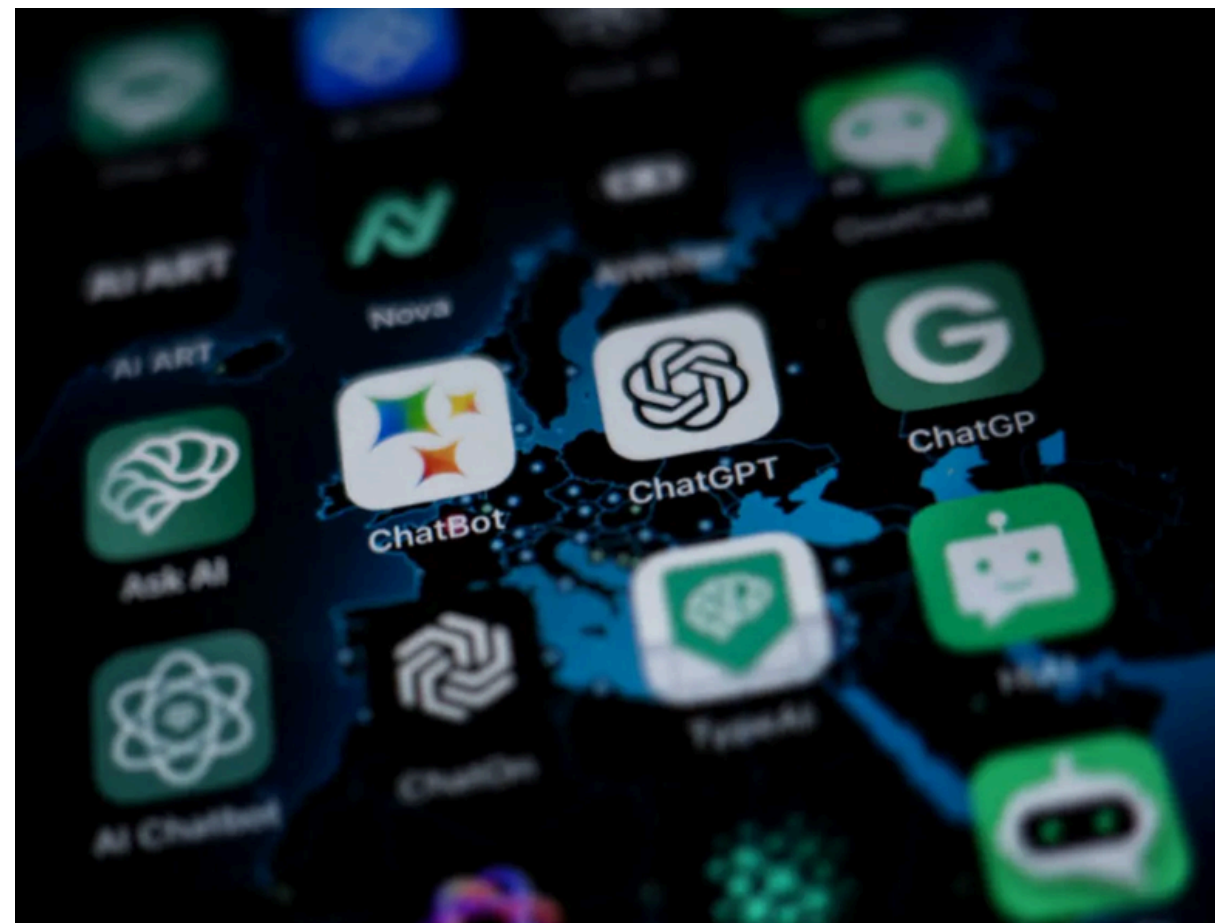


AI ETHICS



ĐẠO ĐỨC TRONG AI

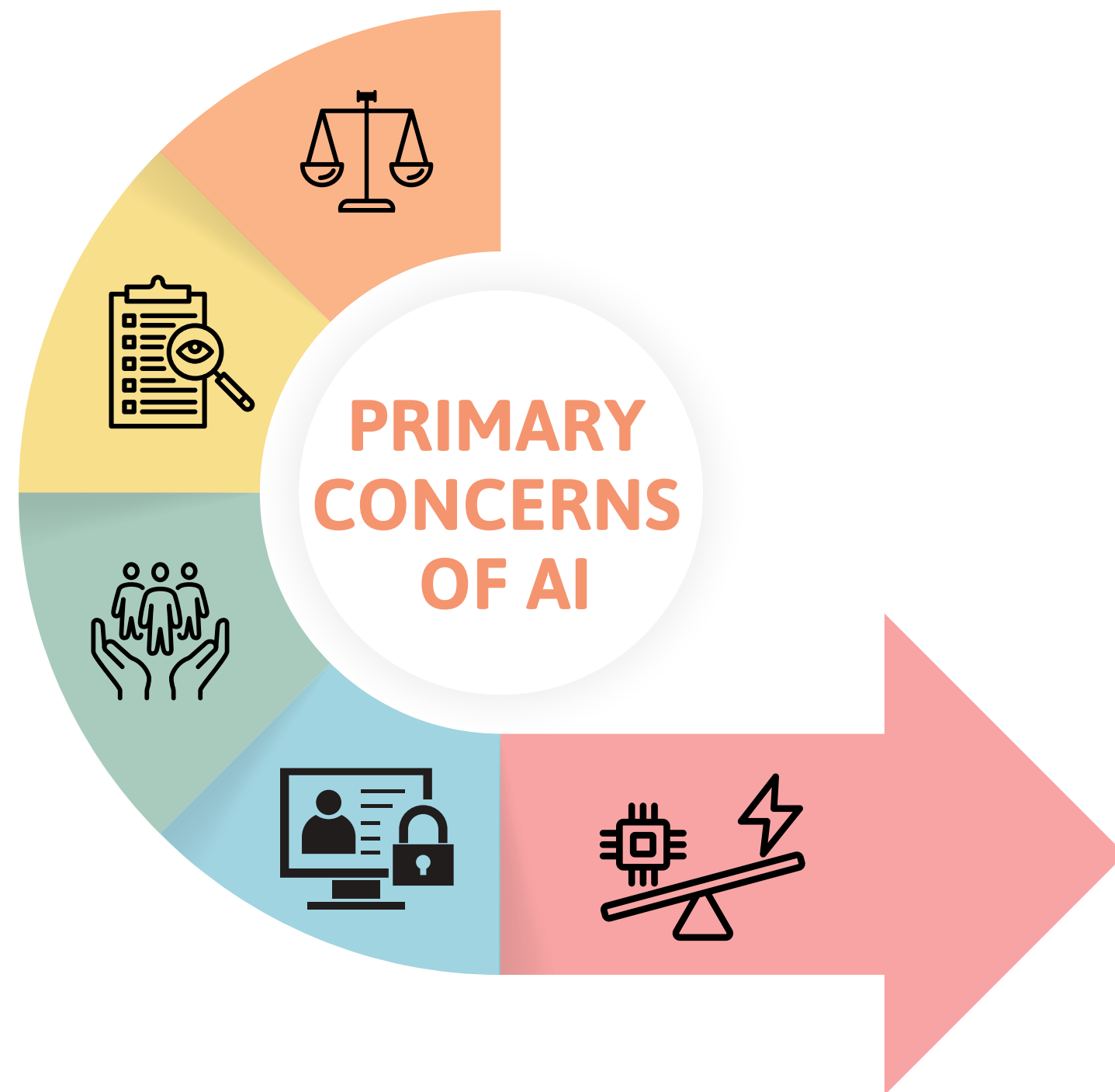
Đạo đức AI: việc áp dụng các khuôn khổ đạo đức vào việc **phát triển** và **sử dụng** trí tuệ nhân tạo



Con người

Making decision
(Ra quyết định)

NHỮNG MỐI QUAN TÂM CHÍNH VỀ AI HIỆN NAY



- 01** Fairness
(Công bằng)
- 02** Transparency
(Minh bạch)
- 03** Accountability
(Trách nhiệm)
- 04** Privacy Protection
(Bảo vệ quyền riêng tư)
- 05** Bias Mitigation
(Ngăn ngừa sự thiên vị)

NGUYÊN TẮC ĐẠO ĐỨC TRONG XÂY DỰNG AI

Minh Bạch

Việc cung cấp thông tin rõ ràng về cách AI được huấn luyện, dữ liệu đầu vào nào được sử dụng, và cách các quyết định của AI được đưa ra

Trách Nhiệm và Giải Trình

Các tổ chức phát triển và triển khai AI cần phải chịu trách nhiệm về cách AI hoạt động và các tác động của nó

Công Bằng

AI cần tránh thiên vị hoặc tạo ra sự bất công đối với bất kỳ cá nhân hay nhóm nào

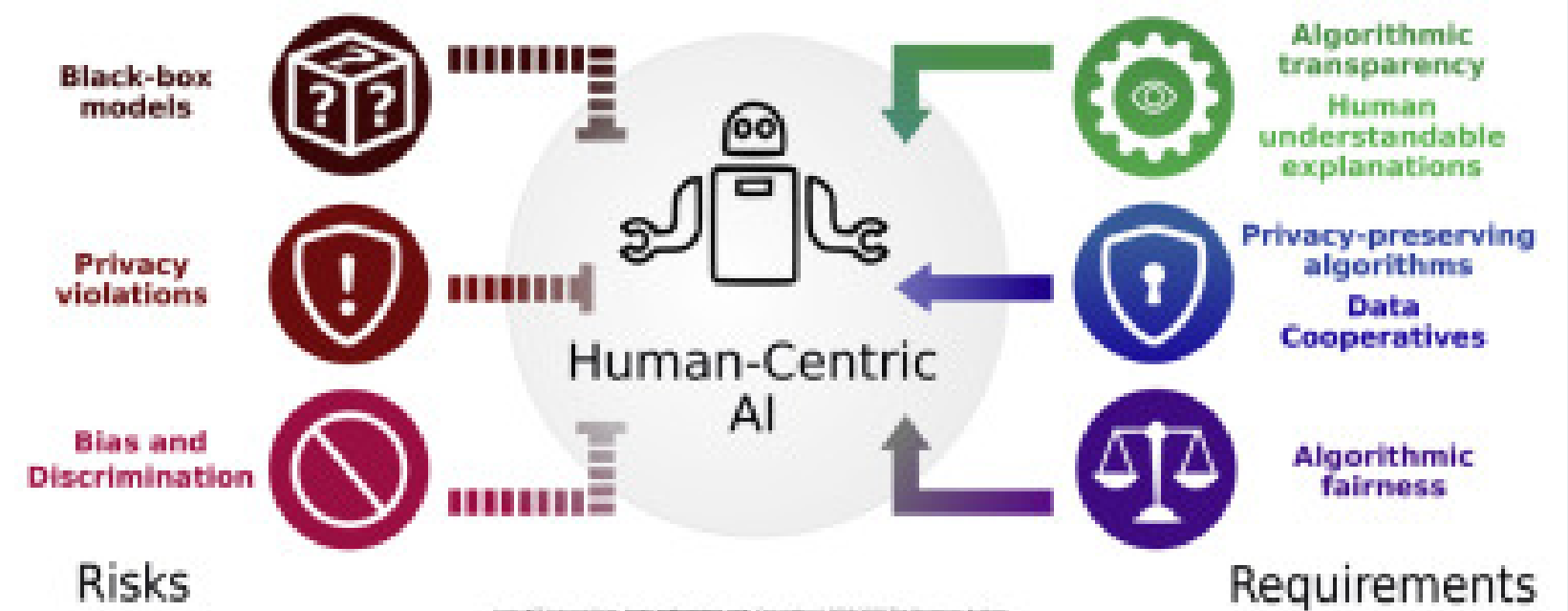
Lợi ích của AI đạo đức đối với xã hội và doanh nghiệp

- Tăng cường niềm tin
- Thúc đẩy phúc lợi xã hội
- Tạo nên uy tín doanh nghiệp



Rủi ro và hậu quả của các hoạt động AI phi đạo đức

- Phân biệt đối xử
- xâm phạm quyền riêng tư
- Vi phạm pháp luật và bị phạt



CẢM ƠN THẦY

VÌ ĐÃ LẮNG NGHE

